

逻辑结构

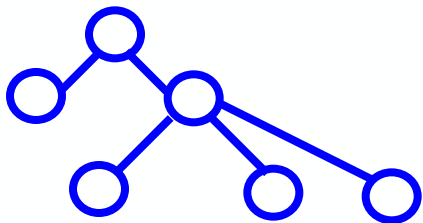
集合——数据元素间仅同属一集合，无其他关系



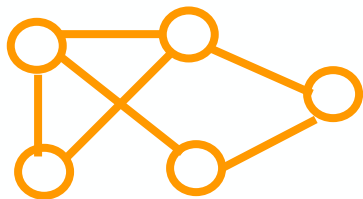
线性结构——一个对一个，如线性表、栈、队列、数组、字符串



树形结构——一个对多个，如树



图状结构——多个对多个，如图



教学目标

1. 掌握树和二叉树的基本概念、性质和存储结构
2. 熟练掌握二叉树的先、中、后序遍历方法
3. 了解线索化二叉树的思想
4. 了解：森林与二叉树的转换
5. 熟练掌握：哈夫曼树的实现方法、构造哈夫曼编码的方法

第5章 树和二叉树 (P137)

5.1 树的定义和基本术语

5.3 树的抽象数据类型定义

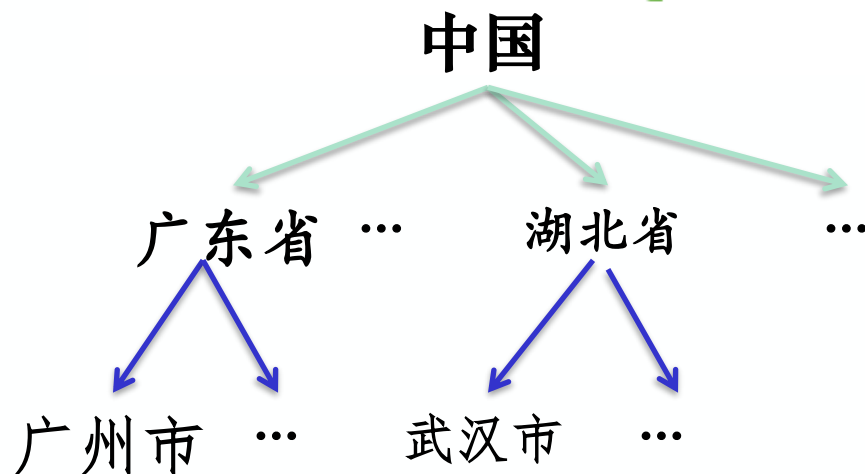
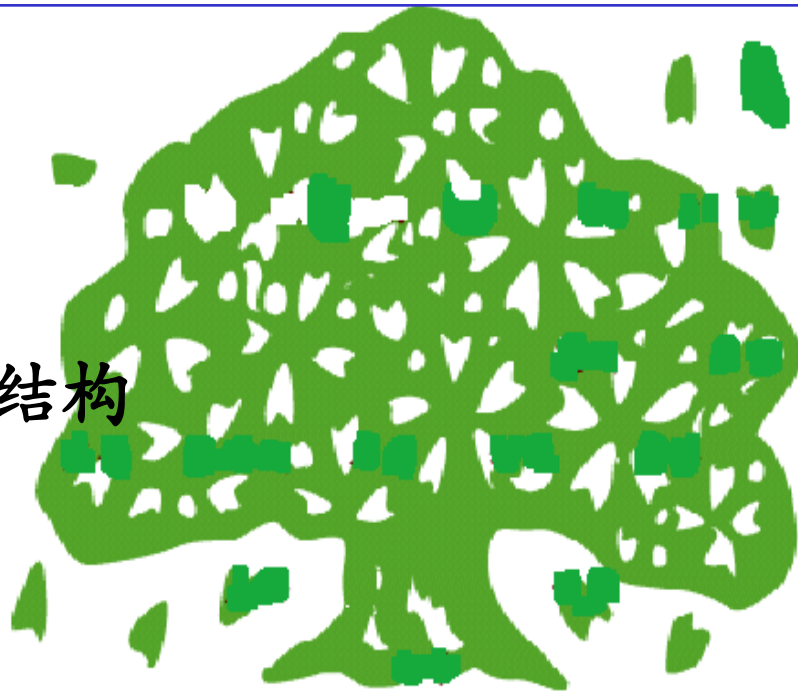
5.4 二叉树的定义、性质和存储结构

5.5 遍历二叉树和线索二叉树

5.6 树和森林

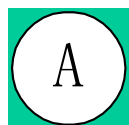
5.7 哈夫曼树及其应用

家谱、文件目录

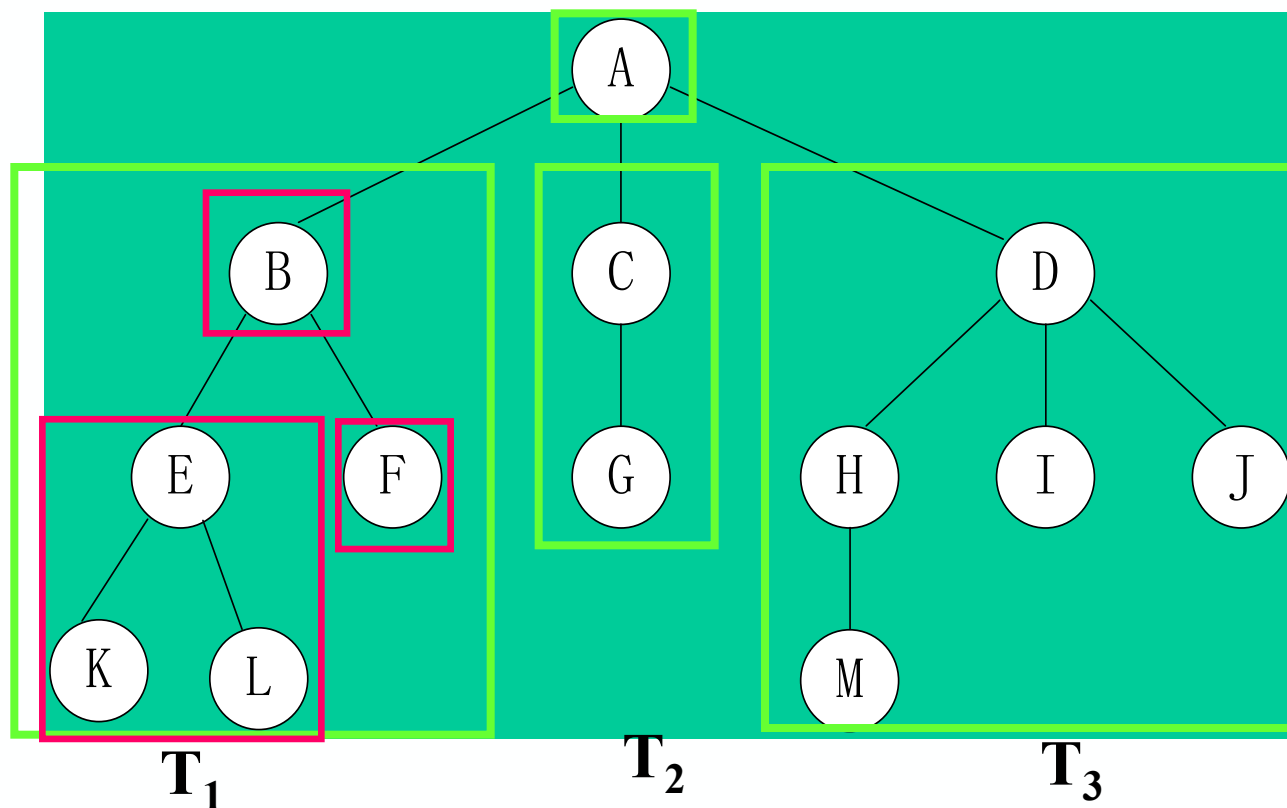


5.1.1 树的定义(P137)

树是 n ($n \geq 0$) 个结点的**有限集**，分为空树和非空树。



只有根结点的树



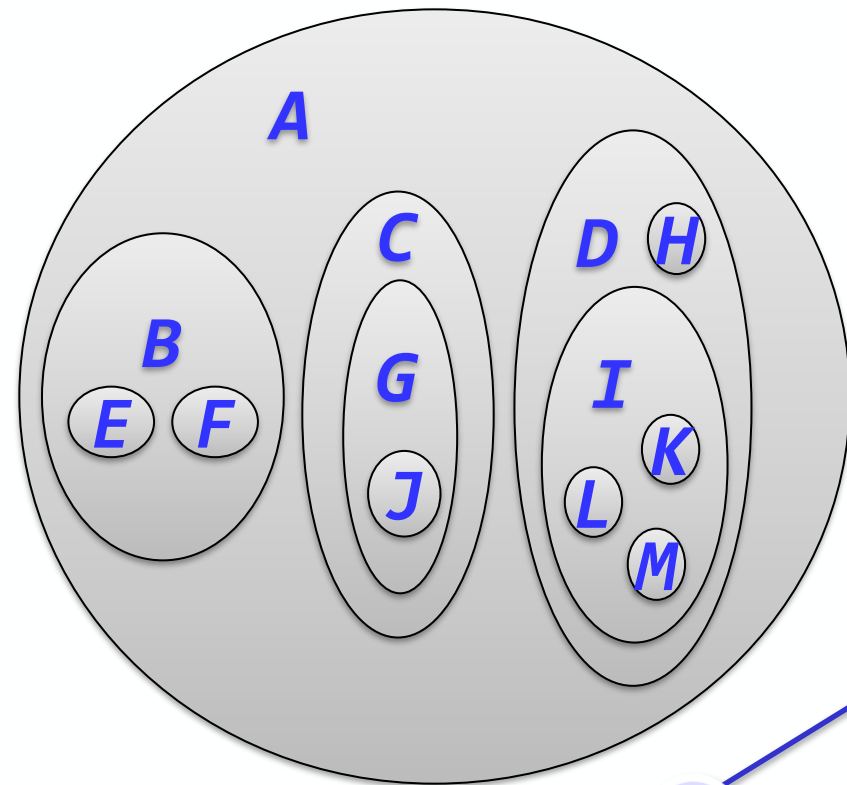
对于非空树 T :

(1) 有且仅有一个称之为**根**的结点;

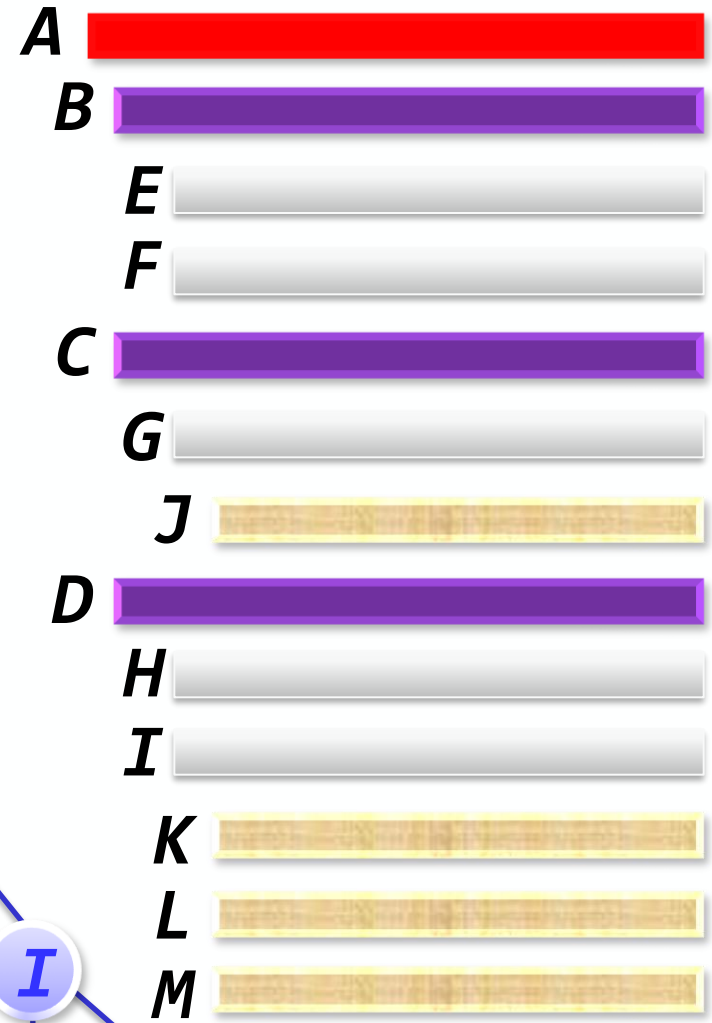
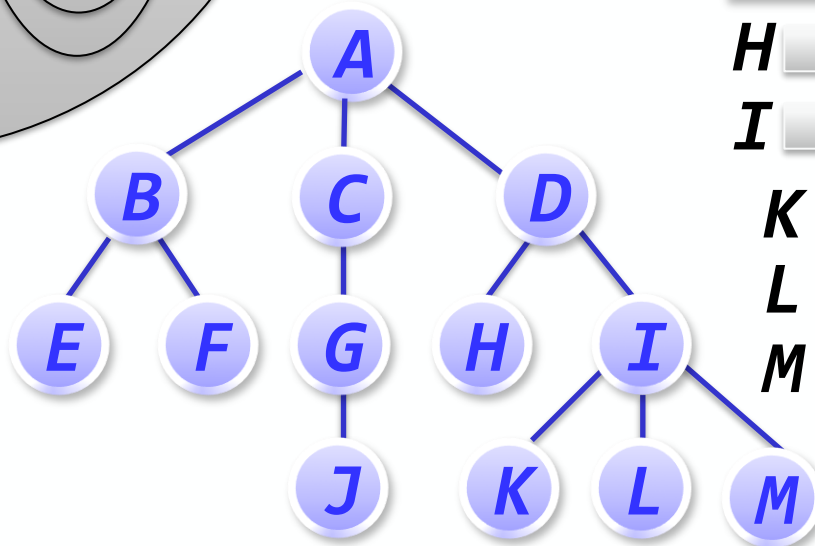
(2) 除根结点以外的其余结点可分为 m ($m > 0$) 个互不相交的**有限集** $T_1, T_2, \dots, T_m$, 其中每一个集合本身又是一棵树, 称为根结点的**子树**。

⇒ 树是一个递归的定义! 树中所有结点构成一种分支层次关系。

5.1.1 树的表示形式(P138)



嵌套集合



凹入表示法



广义表: $(A(B(E, F), C(G(J)), D(H, I(K, L, M))))$

5.1.2 树的基本术语(P138)

结点

——树中的一个独立单元，如A、B、C、D、E...

根

——即根结点，如A

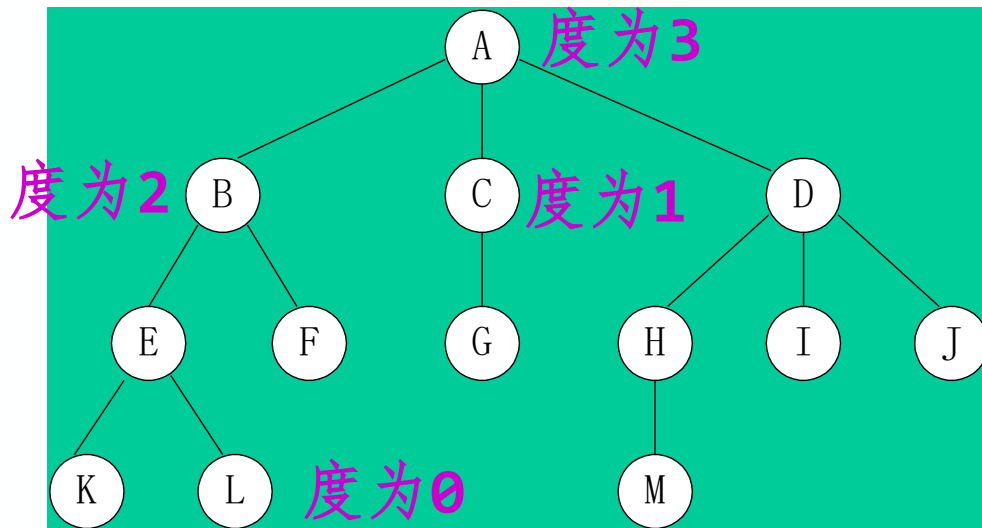
结点的度

——结点拥有的子树数，如A的度为3，C的度为1，F的度为0

树的度

——树内所有结点的度的最大值，如树的度为3

通常将度为m的树称为**m叉树**或者**m次树**



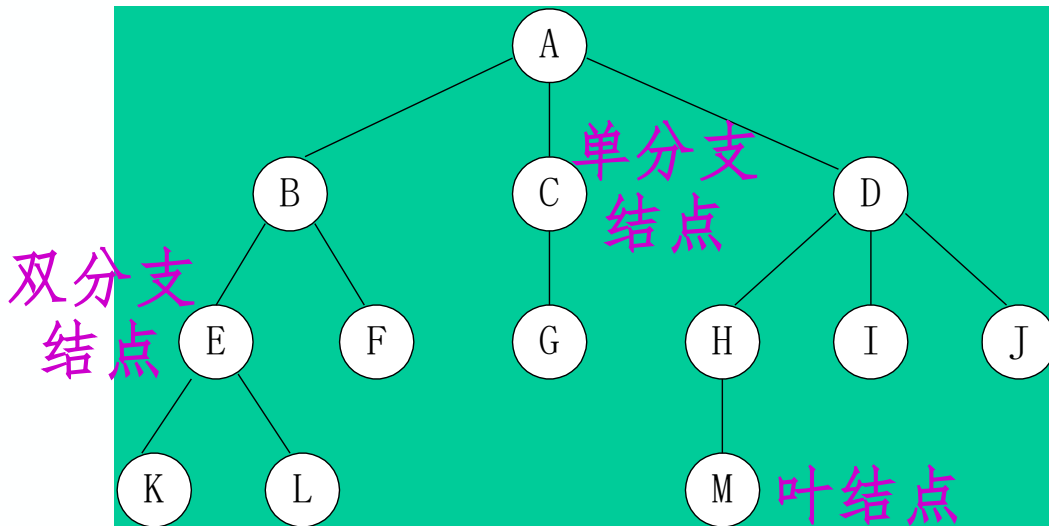
树的度为3

特点：

- ①树的根结点没有前驱结点，除根结点之外的所有结点**有且仅有一个前驱结点**；
- ②树中所有结点**有零个或多个后继结点**。

5.1.2 树的基本术语(P138)

- 叶子结点** —— 度为0的结点，即终端结点(无后继)，如K,L,F,G,M,I,J
- 分支结点** —— 度不为0的结点，称为非终端结点，度为1的结点是**单分支结点**；度为2的结点是**双分支结点**，依此类推
- 孩子结点** —— 在一棵树中，每个结点的后继被称作该结点的孩子结点。相应地，该结点被称作孩子结点的双亲结点
- 兄弟结点** —— 具有同一双亲的孩子结点互为兄弟结点
- 堂兄弟结点** —— 双亲在同一层的结点互为堂兄弟结点



A的孩子结点有B、C、D

B、C、D的双亲结点是A

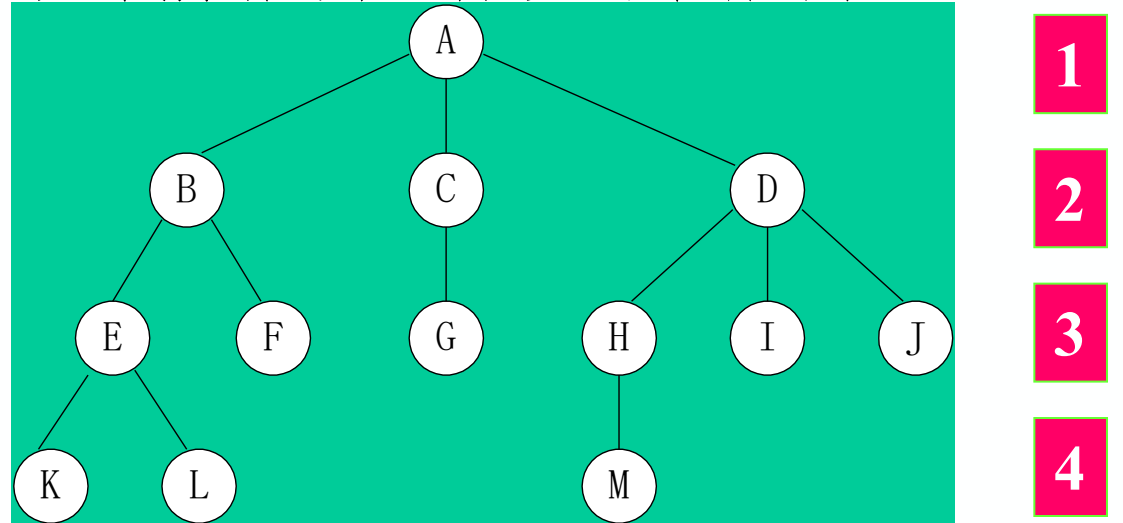
B、C、D互为兄弟结点

E、G、H互为堂兄弟结点

5.1.2 树的基本术语(P138)

- 子孙结点** —— 在一棵树中，一个结点的所有子树中的结点
- 祖先结点** —— 从根结点到到达一个结点的分支上经过的所有结点
- 结点的层次** —— 结点的层次从根开始定义，根结点为第1层，它的孩子结点为第2层，以此类推，任一结点所在的层次为其双亲结点所在的层次加1。
- 树的深度** —— 树中结点的最大层次，如下面树的深度为4

其余所有结点都是A的子孙结点



M的祖先结点为A、D、H

5.1.2 树的基本术语(P138)

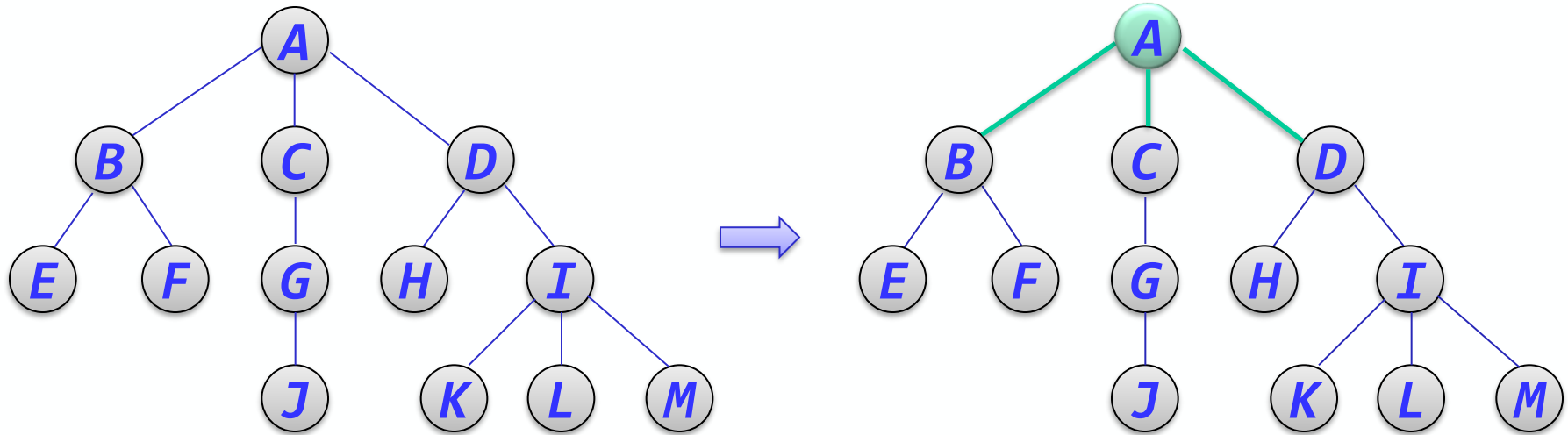
有序树

无序树

——若树中各结点的子树是**按照一定的次序从左向右**安排的，且相对次序是不能随意变换的，则称为**有序树**，否则称为**无序树**。

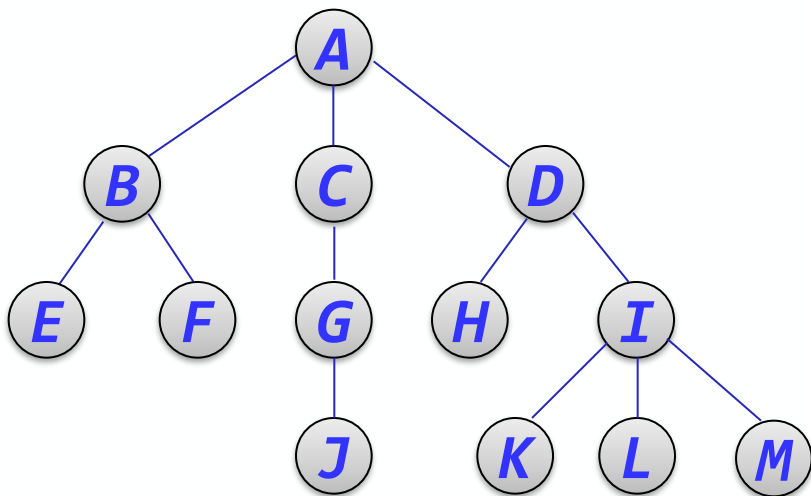
森林

——把含有多棵子树的树的**根结点删去**就成了森林。
 m ($m \geq 0$) 个互不相交的树的集合称为**森林**。



树的性质

性质： 树中的结点数-1 等于 所有结点的度数之和。



- 分支数 = 结点数 - 1
- 结点的度之和 = 分支数
- 结点数-1=结点的度之和
- 度为1的结点：C、G
- 度为2的结点：B、D
- 度为3的结点：A、I

■ 度为 m 的树中，假设 n 为总结点个数， n_i 为度为 i ($0 \leq i \leq m$) 的结点个数： $n = n_0 + n_1 + \dots + n_m$

■ 在所有结点度之和中，一个度为1的结点贡献1个度，一个度为 i 的结点贡献 i 个度， $\dots$ ，一个度为 m 的结点贡献 m 个度

$\Rightarrow$ 所有结点度之和 = $0n_0 + 1n_1 + 2n_2 + \dots + mn_m = n - 1$

【练习1】 一棵度为4的树T中，若有20个度为4的结点，10个度为3的结点，1个度为2的结点，10个度为1的结点，则树T的叶子结点个数是（ ）

A. 41

B. 82

C. 113

D. 122

答案： B

解释： 假设度为 i 的结点有 n_i 个，总结点数

$$n = n_0 + n_1 + n_2 + n_3 + n_4 = n_0 + 10 + 1 + 10 + 20 = n_0 + 41$$

$$n - 1 = \text{度之和} = 0n_0 + 1n_1 + 2n_2 + 3n_3 + 4n_4 = 122, \text{ 得 } n = 123$$

$$n_0 = n - 41 = 123 - 41 = 82$$

【练习2】一棵度为3的树中，度为3、2、1的结点个数分别为3、0、4，则该树中的叶子结点数是（ ）

A. 6

B. 7

C. 8

D. 不可能有这样的树

答案：B

解释：总结点数 $n = n_0 + n_1 + n_2 + n_3 = n_0 + 4 + 0 + 3 = n_0 + 7$

$n - 1 = \text{度之和} = 1n_1 + 2n_2 + 3n_3 = 13$ ，得 $n = 14$

$$n_0 = n - 7 = 7$$

总结点数 = 不同度的结点个数之和 = 度之和 + 1

【练习3】一棵度为4的树中，其度为0、1、2、3的结点数分别为14、4、3、2，求该树的结点总数 n 和度为4的结点个数为（）

答案：25、2

解释：假设度为 i 的结点有 n_i 个，总结点个数为 n

$$n = n_0 + n_1 + n_2 + n_3 + n_4 = 14 + 4 + 3 + 2 + n_4 = 23 + n_4$$

$$n - 1 = 1n_1 + 2n_2 + 3n_3 + 4n_4 = 4 + 6 + 6 + 4n_4 = 16 + 4n_4$$

$$n_4 = 2 \quad n = 25$$

【练习4】 已知一棵度为 m 的树中，有 n_1 个度为1的结点， $\dots$ ，有 n_m 个度为 m 的结点，问该树有多少个叶子结点？

答案： $n_0 = n_2 + 2n_3 + \dots + (m-1)n_m + 1$

解释： 假设总结点个数为 n

$$n = n_0 + n_1 + n_2 + \dots + n_m$$

$$n = 1n_1 + 2n_2 + \dots + mn_m + 1$$

$$1n_2 + 2n_3 + \dots + (m-1)n_m + 1 - n_0 = 0$$

$$n_0 = 1n_2 + 2n_3 + \dots + (m-1)n_m + 1$$

【总结】 综合以上几题：

① **总结点数** = $n_0 + n_1 + n_2 + \dots + n_m$

② **总分支数** = $0 * n_0 + 1 * n_1 + 2 * n_2 + \dots + m * n_m$

③ **总结点数** = **总分支数** + 1 = **度之和** + 1

④ **叶子结点数** = $n_2 + 2 * n_3 + \dots + (m-1) * n_m + 1$

第5章 树和二叉树 (P110)

5.1 树的定义和基本术语

5.3 树的抽象数据类型定义

5.4 二叉树的定义、性质和存储结构

5.5 遍历二叉树和线索二叉树

5.6 树和森林

5.7 哈夫曼树及其应用



5.3 树的抽象数据类型定义(P141)

ADT Tree

{ 数据对象D: D是具有相同特性的数据元素的集合。

数据关系R: 若D为空集, 则称为空树; //允许 $n=0$

若D中仅含一个数据元素, 则R为空集; //仅有根结点

其他情况下的R存在二元关系:

① root 唯一 //关于根的说明

② $D_j \cap D_k = \Phi$ //子树交集为空, 互不相交

③ //关于数据元素的说明

基本操作 P:

//如求树的深度、某结点的双亲、根结点等

} ADT Tree

基本操作 P:

InitTree(&T) // 初始化置空树

CreateTree(&T, definition) // 按定义构造树T

Root(T) // 若树T存在，返回树的根结点

Value(T, cur_e) // 返回当前结点cur_e的元素值

Parent(T, cur_e) // 求当前非根结点cur_e的双亲结点

RightSibling(T, cur_e) // 求当前结点cur_e的右兄弟

LeftChild(T, cur_e) // 求当前非叶结点cur_e的最左孩子

TreeEmpty(T) // 判定树T是否为空树，返回true/false

TreeDepth(T) //若树T存在，返回树的深度

TraverseTree(T, Visit()) // 若树T存在，按某种次序遍历

基本操作 P:

Assign(T, cur_e, value) // 给当前结点cur_e赋值为value

ClearTree(&T) //若树T存在，将树清空

DestroyTree(&T) //若树T存在，销毁树的结构

InsertChild(&T, p, i, c) // 将以c为根的树插入为树T中结点p的第i棵子树

DeleteChild(&T, p, i) // 删除树T中结点p的第i棵子树

第5章 树和二叉树 (P144)

5.1 树的定义和基本术语

5.3 树的抽象数据类型定义

5.4 二叉树的定义、性质和存储结构

5.5 遍历二叉树和线索二叉树

5.6 树和森林

5.7 哈夫曼树及其应用



为何要重点研究每结点最多只有两个“叉”的树？

- ✓结构简单，存储效率高，二叉树的操作算法相对简单且任何树都很容易转化成唯一对应的二叉树结构。

普通树（多叉树）如果不转化为二叉树，运算很难实现。

二叉树与树的区别主要有以下两点：

- ✓二叉树中结点**至多**有两棵子树（不存在度大于2的结点）
- ✓二叉树的子树**有左右之分**，其次序不能颠倒

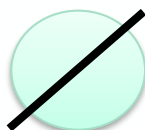
5.1.3 二叉树的定义(P139)

➤ 二叉树是 n ($n \geq 0$) 个结点所构成的集合，它或为空树 ($n = 0$) 或为非空树。

递归
定义

➤ 对于非空树 T ： (1) 有且仅有一个称之为根的结点；
(2) 除根结点以外的其余结点分为两个互不相交的子集 T_1 和 T_2 ，分别称为 T 的左子树和右子树，且 T_1 和 T_2 本身又都是二叉树。

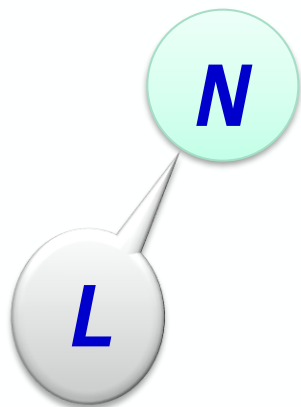
①空树



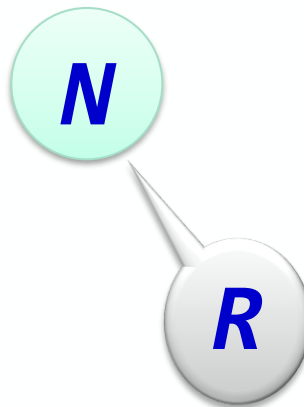
②只含根结点



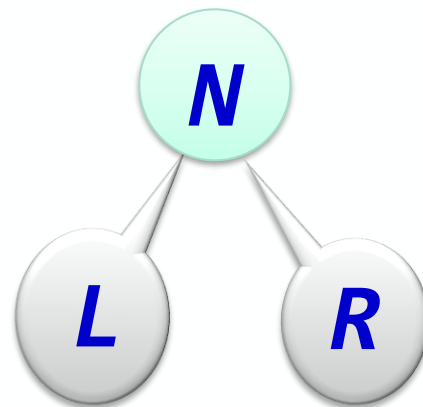
③右子树为空树



④左子树为空树



⑤左右子树均不为空树

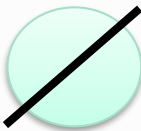


二叉树的五种基本形态(P139)

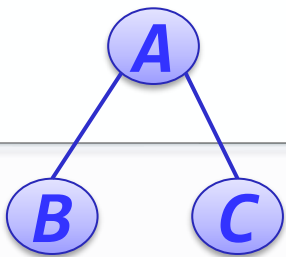
二叉树和度为2的树有什么区别？

■ 度为2的树至少有3个结点；

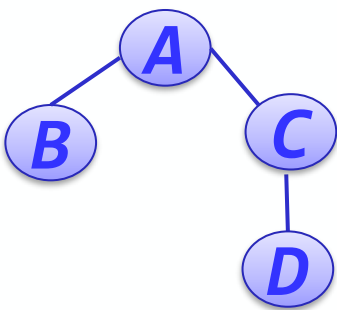
二叉树可以是空树，结点个数可以为0。



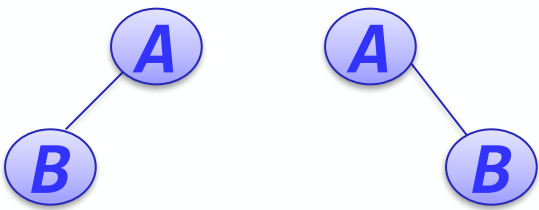
■ 度为2的树中，度为1的结点的孩子结点不分左、右；
而二叉树中度为1的结点的孩子需要区分左、右孩子



度为2的树



树中度为1结点，不区分左右

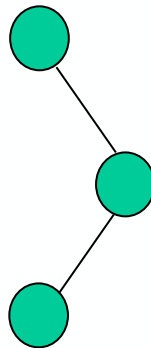
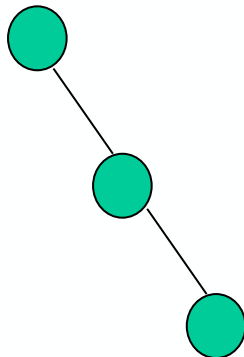
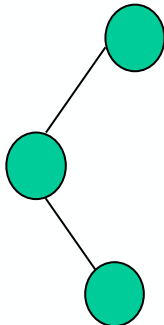
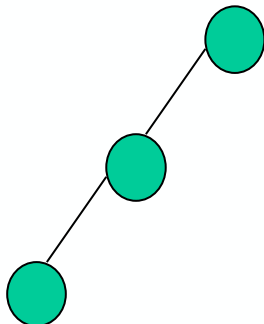
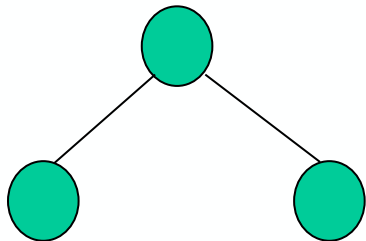


二叉树中度为1结点，区分左右子树

思考

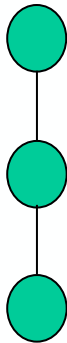
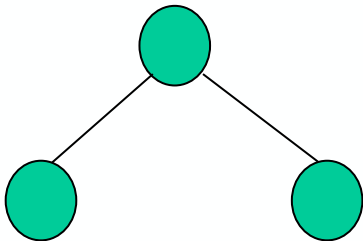
具有3个结点的二叉树可能有几种不同形态?

5种



具有3个结点的普通树可能有几种不同形态?

2种



【练习5】一棵二叉树共有25个结点，其中5个是叶子结点，则度为1的结点数是（ ），度为2的结点数是（ ）

A. 10

B. 16

C. 6

D. 4

答案：B、D

解释：总结点数 $25 = n_0 + n_1 + n_2 = 5 + n_1 + n_2$

$n - 1 = \text{度之和} = n_1 + 2n_2$ ，得 $n_1 + 2n_2 + 1 = 5 + n_1 + n_2$

$n_2 = 4$ ， $n_1 = 16$

【练习6】 设某二叉树中共有140个结点，其中40个度为1的结点，则（ ）

- A. 不可能有这样的二叉树
- B. 该二叉树有51个度为2的结点
- C. 该二叉树有50个叶子结点
- D. 该二叉树中有50个度为2的结点

答案：A

解释：总结点数 $140 = n_0 + n_1 + n_2 = 40 + n_0 + n_2$

$n - 1 = \text{度之和} = 40 + 2n_2$ ，得 $40 + 2n_2 + 1 = 140$

$n_2 = 49.5$ ， $n_0 = 50.5$ 不存在

5.3 二叉树的抽象数据类型定义(P142)

ADT BinaryTree

{ 数据对象D: D是具有相同特性的数据元素的集合。

数据关系R: 若 $D=\Phi$, 则 $R=\Phi$; //BinaryTree是空二叉树

若 $D\neq\Phi$, 则 $R=\{H\}$; 存在二元关系:

① root 唯一 //关于根的说明

② $D_j \cap D_k = \Phi$ //子树交集为空, 互不相交

③ //关于数据元素的说明

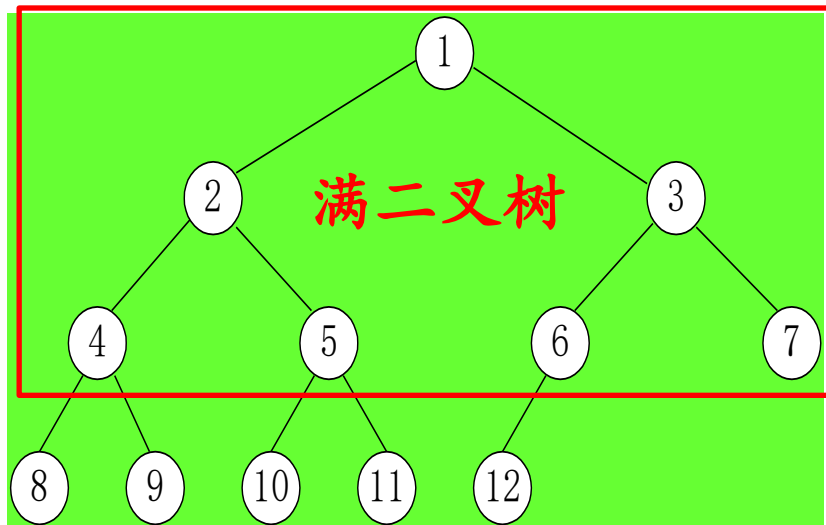
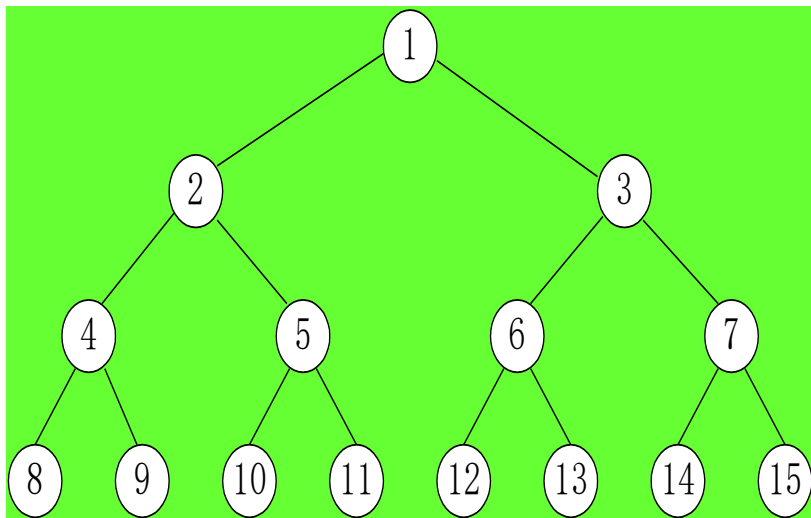
④ //关于左子树和右子树的说明

基本操作 P:

//构造二叉链表, 先序、中序、后序遍历等

} ADT BinaryTree

特殊形态的二叉树(P145)



满二叉树：一棵深度为 k 且有 $2^k - 1$ 个结点的二叉树。

(特点：每层都“充满”了结点)

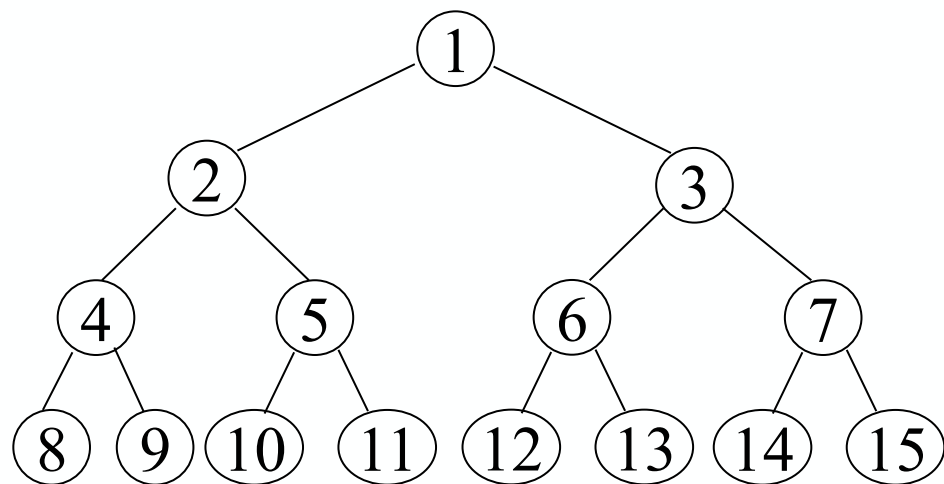
$$k = 4$$

$$n = 2^4 - 1 = 15$$

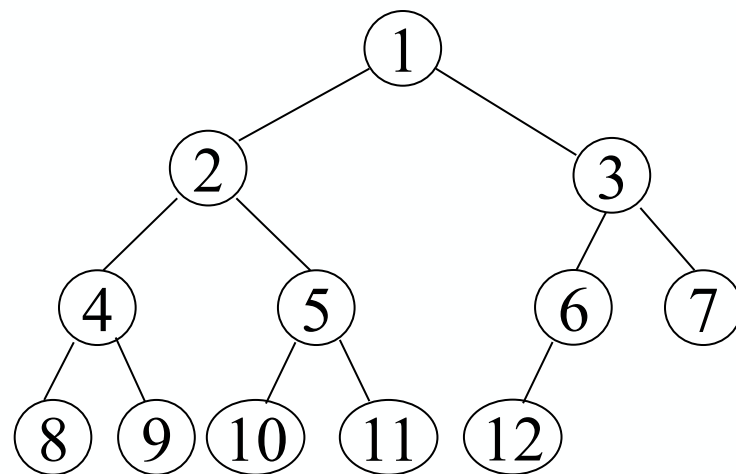
完全二叉树：最多只有下面两层的结点的度数小于2，最下面一层的叶结点都依次排列在该层最左边的位置上。

完全二叉树实际上是对应的**满二叉树**连续地删除叶结点层最右边若干个结点得到的

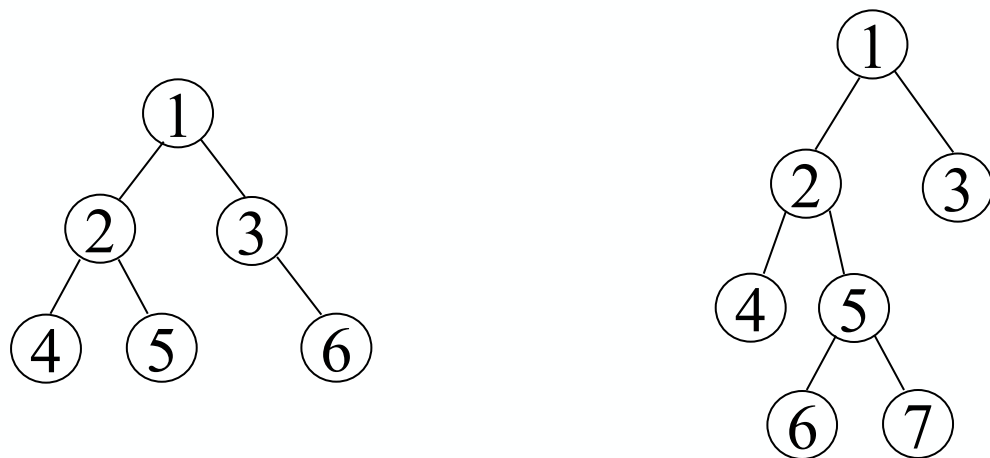
特殊形态的二叉树(P145)



(a) 满二叉树



(b) 完全二叉树



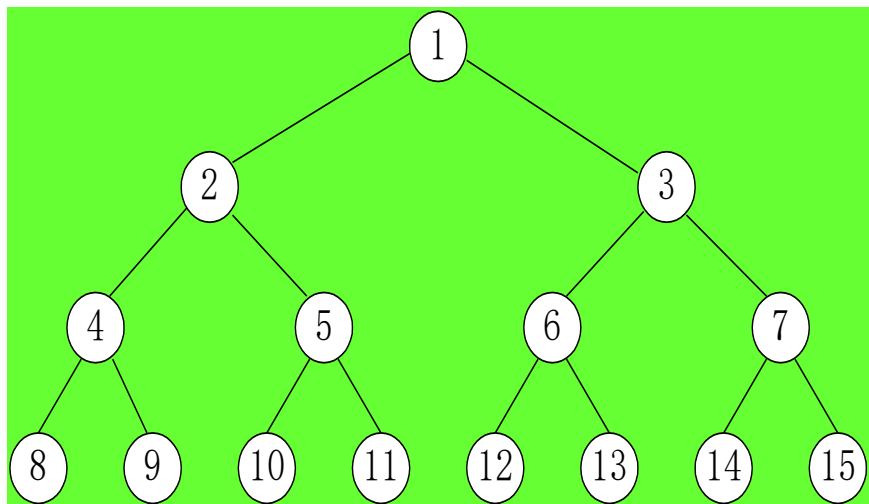
(c) 非完全二叉树

书中定义：深度为 k 的具有 n 个结点的二叉树，当且仅当其每一个结点都与深度为 k 的满二叉树中编号为 $1\sim n$ 的结点一一对应时，称之为完全二叉树。

满二叉树是完全二叉树的一个特例

5.4 二叉树的性质和存储结构(P144)

性质1: 在二叉树的第*i*层上**至多**有 2^{i-1} 个结点 ($i \geq 1$)



当*i*=1层, 至多有 $1=2^{1-1}$

当*i*=2层, 至多有 $2=2^{2-1}$

当*i*=3层, 至多有 $4=2^{3-1}$

当*i*=4层, 至多有 $8=2^{4-1}$

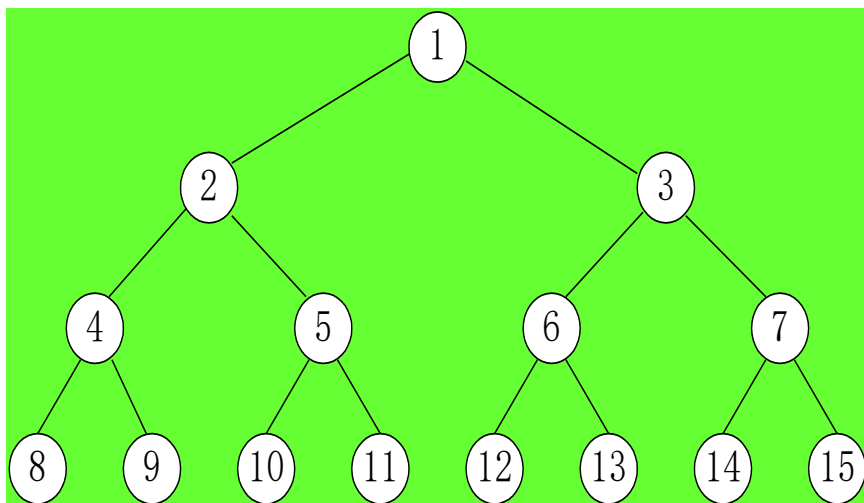
.....

当*i*=*i*层, 至多有 2^{i-1}

提问: 二叉树的第*i*层上至少有 1 个结点?

5.4 二叉树的性质和存储结构(P144)

性质2: 深度为**k**的二叉树**至多**有 $2^k - 1$ 个结点($k \geq 1$)



树的深度K为4

如左图深度**k**为4，至多有
 $2^k - 1 = 15$ 个结点

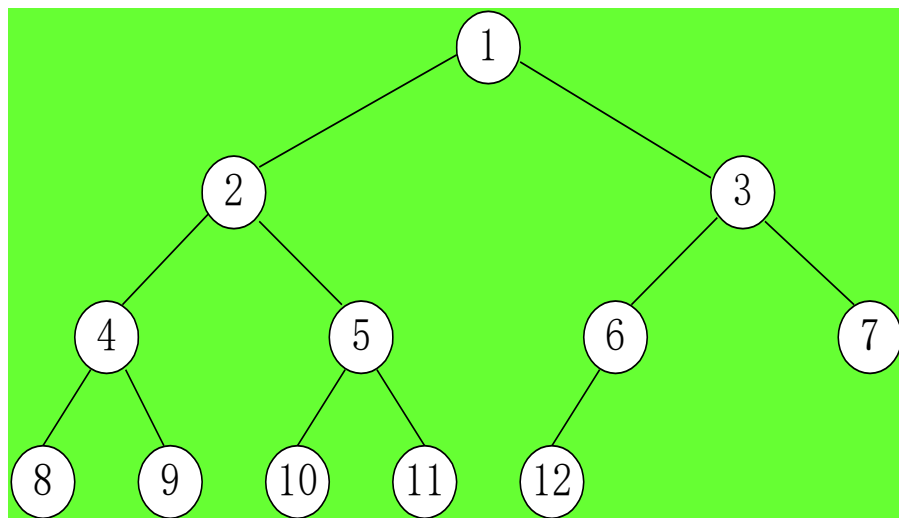
$$S_n = \frac{a_1(1 - q^n)}{1 - q} \quad (q \neq 1)$$

性质1: 在二叉树的第**i**层上**至多**有 2^{i-1} 个结点 ($i \geq 1$)

所以深度为**k**时，至多有 $2^0 + 2^1 + \dots + 2^{k-1} = \frac{2^0(1 - 2^k)}{1 - 2}$
 $= 2^k - 1$ 个结点

5.4 二叉树的性质和存储结构(P144)

性质3: 对于**任何**一棵二叉树T, 若叶子结点数记为 n_0
度为2的结点数记为 n_2 , 则 $n_0 = n_2 + 1$



如左图叶子结点有
8,9,10,11,12,7, 共 $n_0=6$

度为2的结点有1,2,3,4,5
共 $n_2=5$ 即 $n_0 = n_2 + 1$

总结点数 $n = n_0 + n_1 + n_2$

总结点数 $n - 1 = \text{度之和} = n_1 + 2n_2$

$$n_0 + \cancel{n_1} + \cancel{n_2} = \cancel{n_1} + \cancel{2n_2} + 1$$

【练习7】 一棵二叉树中有7个度为2的结点和5个度为1的结点，其总共有（ ）个结点。

A. 16

B. 18

C. 20

D. 30

答案：C

解释： $n_2=7$, $n_1=5 \Rightarrow n_0=8$

$$n=n_0+n_1+n_2=20$$

【练习8】 设二叉树中共有375个结点，其中度为2的结点有187个，则度为1的结点个数是（ ）。

A. 不可能有这样的二叉树

B. 1

C. 188

D. 0

答案： D

解释： $n = n_0 + n_1 + n_2$; $n_0 = n_2 + 1$

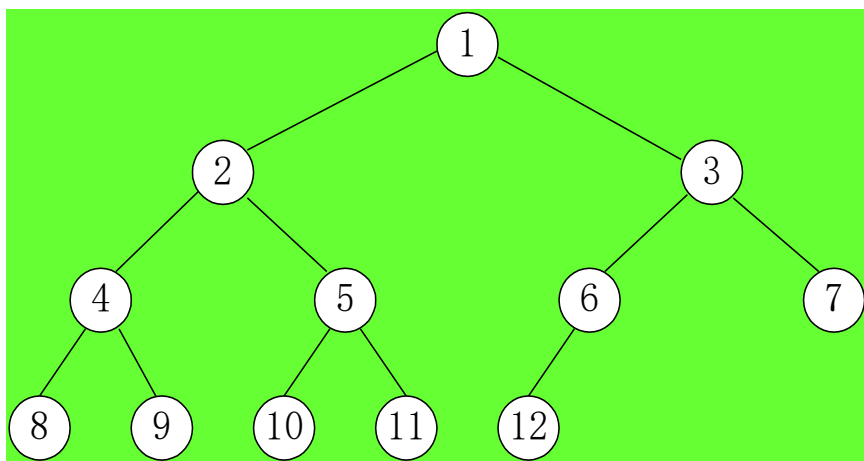
$$\Rightarrow 375 = n_1 + 2n_2 + 1 = n_1 + 375$$

$$n_1 = 0$$

5.4 二叉树的性质和存储结构(P145)

性质4: 具有 n 个结点的完全二叉树的深度为 $\lfloor \log_2 n \rfloor + 1$ 。

$\lfloor x \rfloor$: 向下取整, 表示不大于 x 的最大整数 $\lfloor 3.5 \rfloor = 3$



如左图共 $n=12$ 个结点

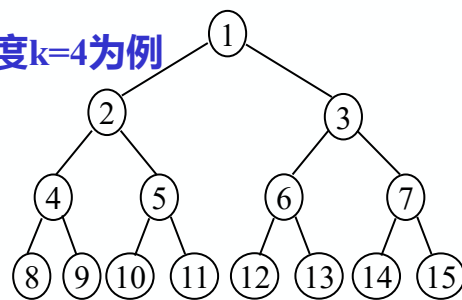
深度 $=\lfloor \log_2 n \rfloor + 1 = 4$

$\log_2 8 < \log_2 12 < \log_2 16$

$3 < \log_2 12 < 4$

性质2: 深度为 k 的二叉树至多有 $2^k - 1$ 个结点($k \geq 1$)

以深度 $k=4$ 为例



假设总结点数 n 最多, 则最后一个结点的编号 $=2^k - 1$

完全二叉树第 k 层结点最少的情况:

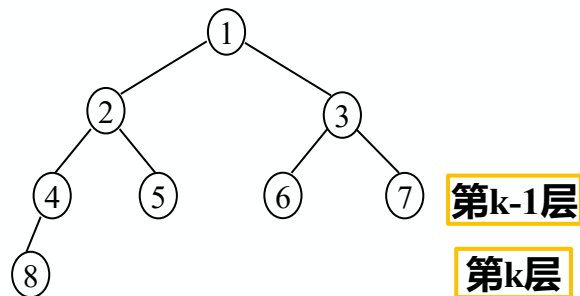
最后一个结点的编号 $n = 2^{k-1} - 1 + 1 = 2^{k-1}$

$$2^{k-1} - 1 < n \leq 2^k - 1$$

$$\text{或 } 2^{k-1} \leq n < 2^k$$

$$k-1 \leq \log_2 n < k$$

$$k = \lfloor \log_2 n \rfloor + 1$$



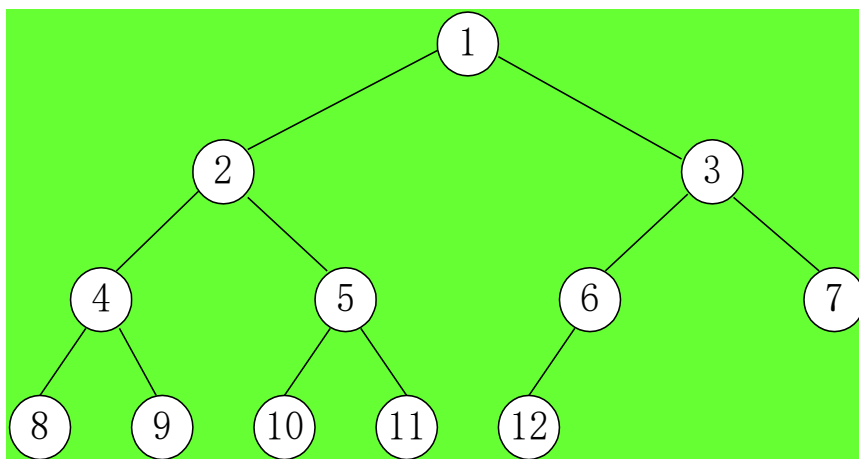
5.4 二叉树的性质和存储结构(P145)

性质5: 如果对一棵有 n 个结点的完全二叉树（深度为 $\lfloor \log_2 n \rfloor + 1$ ）的结点按层序编号（从上至下、从左及右）则对任一结点 i （ $1 \leq i \leq n$ ），有：

如果 $i=1$ ，则结点 i 是二叉树的根，无双亲；如果 $i > 1$ 则其双亲是结点 $\lfloor i/2 \rfloor$ ；

如果 $2i > n$ ，则结点 i 无左孩子（结点 i 为叶子结点）否则其左孩子是结点 $2i$ ；

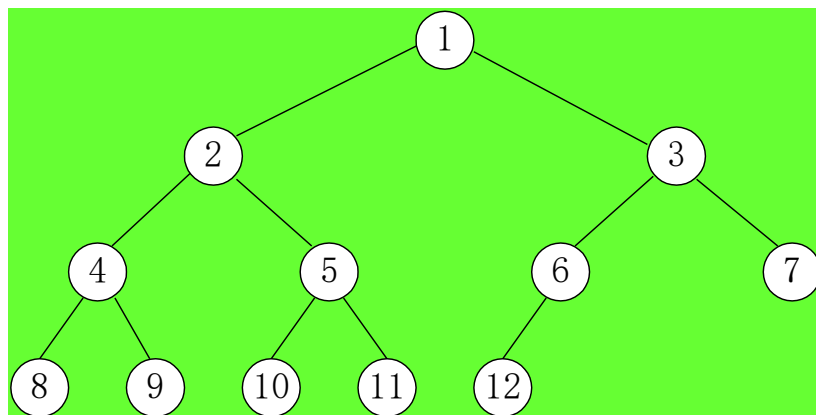
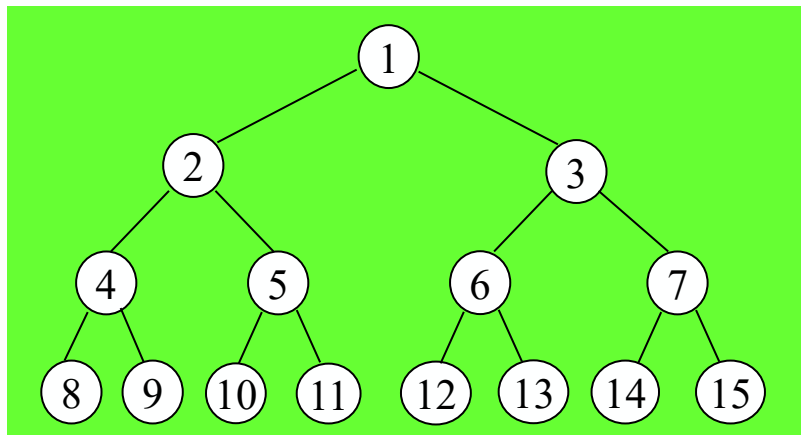
如果 $2i + 1 > n$ ，则结点 i 无右孩子，否则其右孩子是结点 $2i + 1$ ；



5.4 二叉树的性质和存储结构

性质延伸: 如果对一棵有 n 个结点的完全二叉树(深度为 $\lfloor \log_2 n \rfloor + 1$)的结点按层序编号(从上至下、从左及右), 则对任一结点 i 有:

- ① **叶子结点**只可能在**层次最大的两层**上出现, 对于**最大层次中的叶子结点**, 都依次排列在**该层最左边**的位置上;
- ② 如果有度为1的结点, 只可能有一个, 且该结点只有左孩子而无右孩子;
 $n_1 \leq 1$
- ③ 按层序编号后, 一旦出现**某结点(编号为 i)为叶子结点或只有左孩子**, 则编号**大于 i 的结点均为叶子结点**;
- ④ 若 n 为奇数, 则每个分支结点都有左子树和右子树, 若 n 为偶数, 则编号最大的分支结点, 只有左子树, 没有右子树, 其余分支结点左右子树都有。



【练习9】 一棵完全二叉树上有1001个结点，其中叶子结点的个数是（ ）。

A. 250

B. 500

C. 254

D. 501

答案： D

解释： $n_0 = n_2 + 1$, $n = n_0 + n_1 + n_2 = 1001$

⇒ $n_1 + 2n_2 = 1000$, 由完全二叉树的性质可得

$n_1 = 0$ 或 1 , 且 n_2 为整数, 故 $n_1 = 0$, $n_2 = 500$

$n_0 = 501$, 即有501个叶子结点。

【练习10】一棵完全二叉树中有8个叶子结点，则高度至多是（ ）。

A. 3

B. 4

C. 5

D. 不确定

答案：C

解释： $n_0 = n_2 + 1 = 8$ ， $n_2 = 7$ ， $n = n_0 + n_1 + n_2 = 15 + n_1$

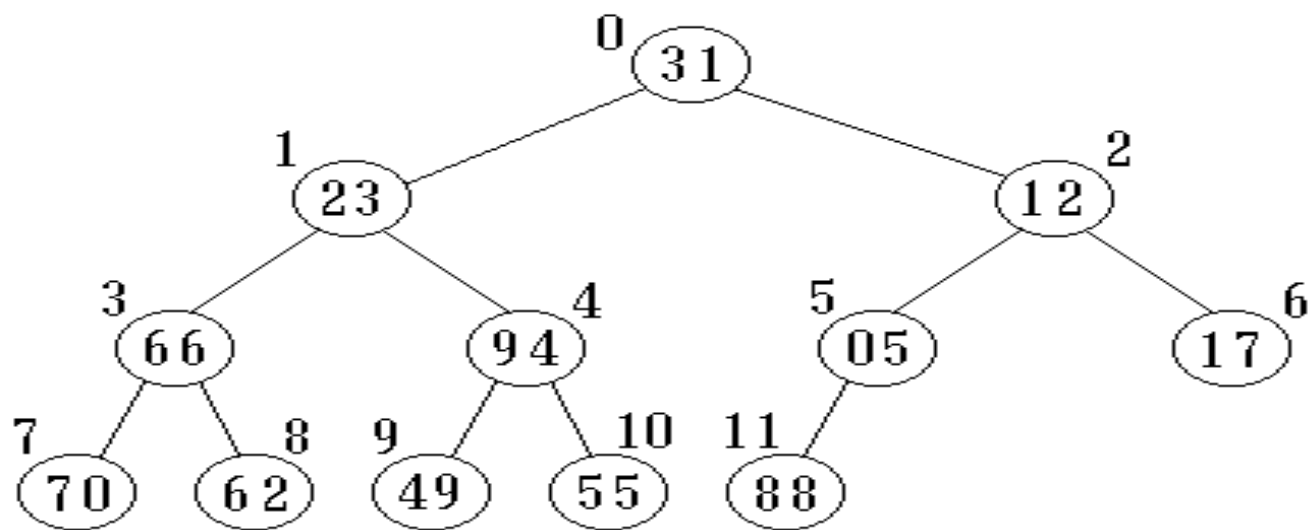
完全二叉树中 $n_1 = 0$ 或 1 ，且 $n_1 = 1$ 时结点个数最多，此时 $n = 16$

⇒ 最大高度 $h = \lfloor \log_2 n \rfloor + 1 = 5$

性质4: 具有 n 个结点的完全二叉树的深度为 $\lfloor \log_2 n \rfloor + 1$ 。

5.4.2 二叉树的顺序存储(P146)

实现：用一组地址连续的存储单元依次“**自上而下、自左至右**”存储完全二叉树的数据元素。



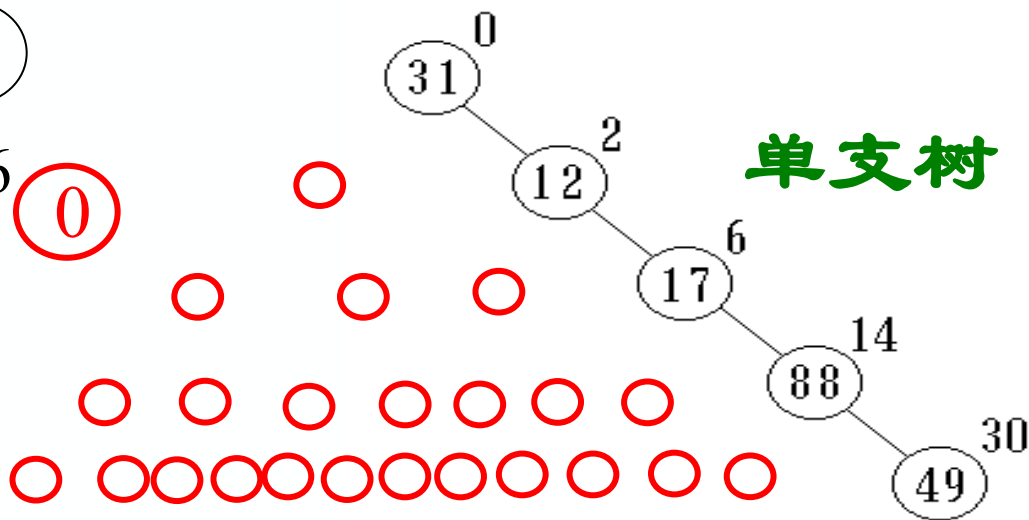
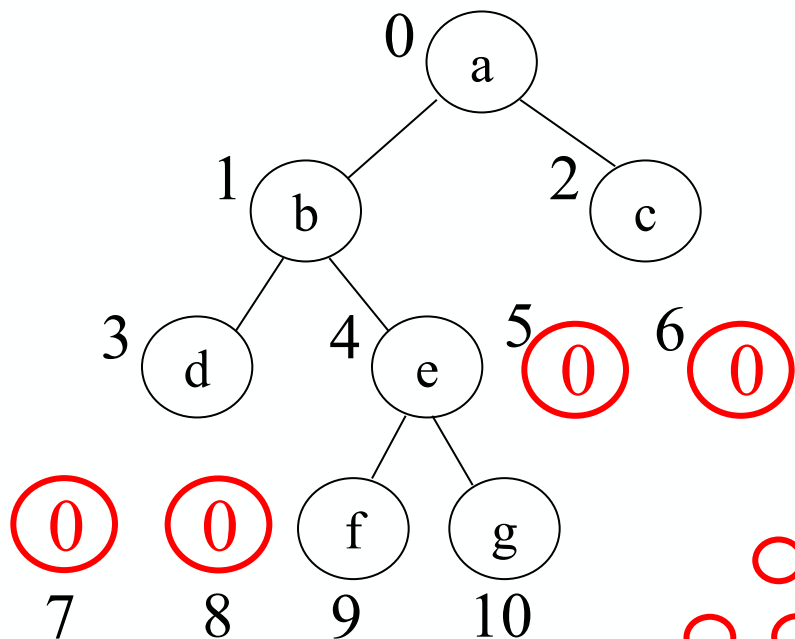
(a)

0	1	2	3	4	5	6	7	8	9	10	11
31	23	12	66	94	05	17	70	62	49	55	88

5.4.2 二叉树的顺序存储(P146)

0	1	2	3	4	5	6	7	8	9	10
a	b	c	d	e	0	0	0	0	f	g

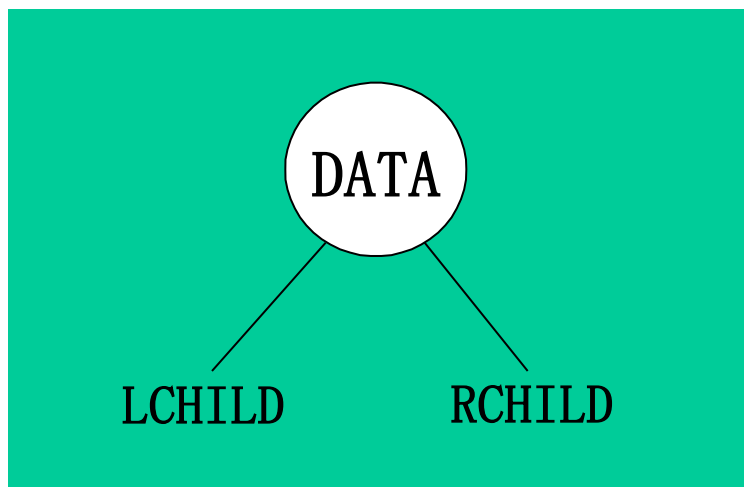
最坏情况下，深度为 k 且只包含 k 个结点的单支树却需要长度为 $2^k - 1$ 的一维数组来存储



特点：适于存满二叉树和完全二叉树，但对于一般二叉树浪费空间。

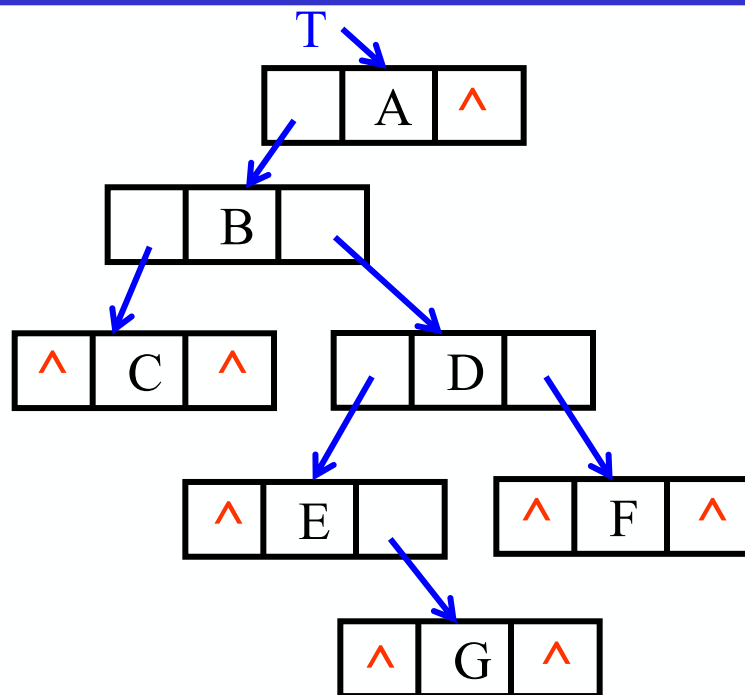
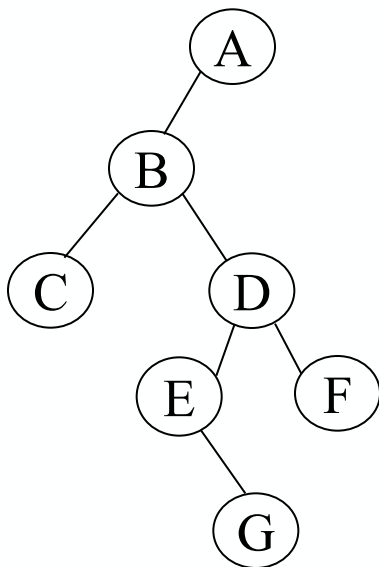
二叉树的链式存储(P146)

二叉树的结点由一个数据元素和分别指向其左、右子树的两个分支构成，则包含三个域：



lchild	data	rchild
--------	------	--------

二叉链表(P146-147)

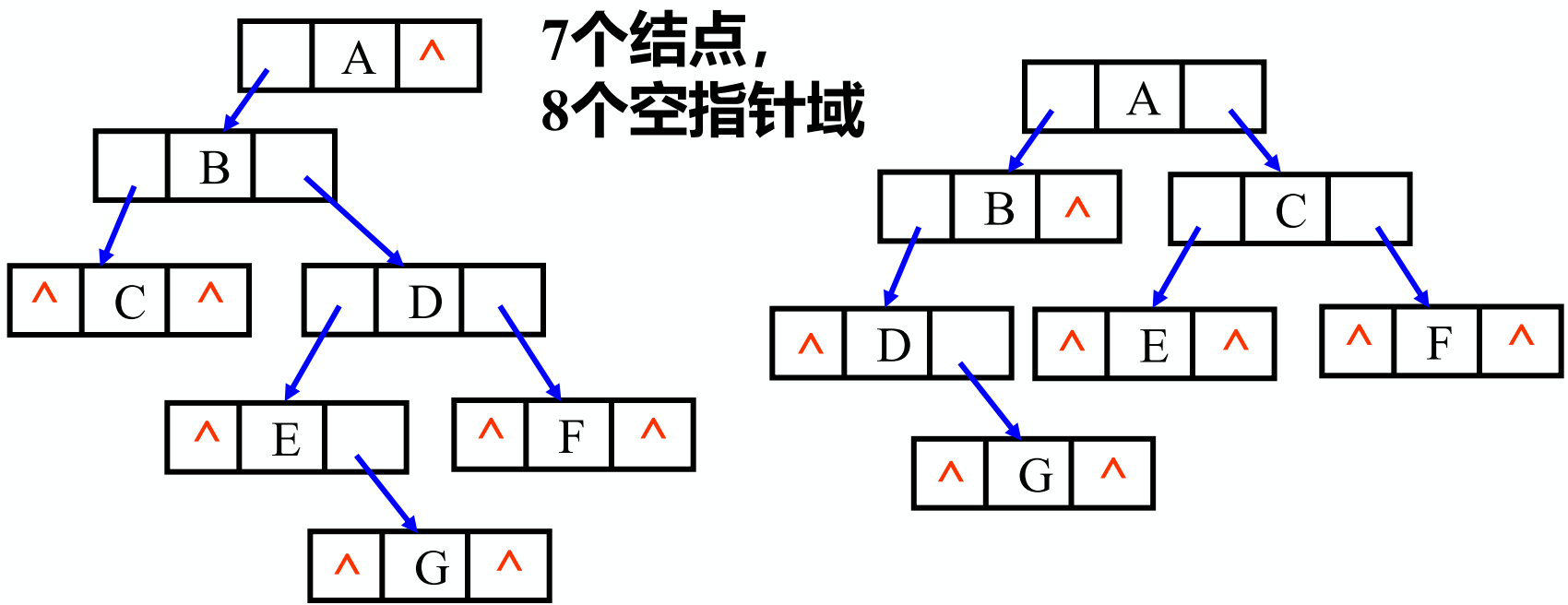


```
typedef struct BiNode{  
    TElemType  data; //结点的数据域  
    struct BiNode  *lchild,*rchild; //左右孩子指针  
}BiNode,*BiTree;  指向的都是二叉结点：递归性
```

BiTree T; //定义指向根结点的指针T

思考

在 n 个结点的二叉链表中，有 $n+1$ 个空指针域



- n 个结点 $\Rightarrow$ $2n$ 个指针域
- 分支数为 $n-1$ $\Rightarrow$ 非空指针域有 $n-1$ 个
- 空指针域个数 = $2n - (n-1) = n+1$

第5章 树和二叉树 (P147)

5.1 树的定义和基本术语

5.3 树的抽象数据类型定义

5.4 二叉树的定义、性质和存储结构

5.5 遍历二叉树和线索二叉树

5.6 树和森林

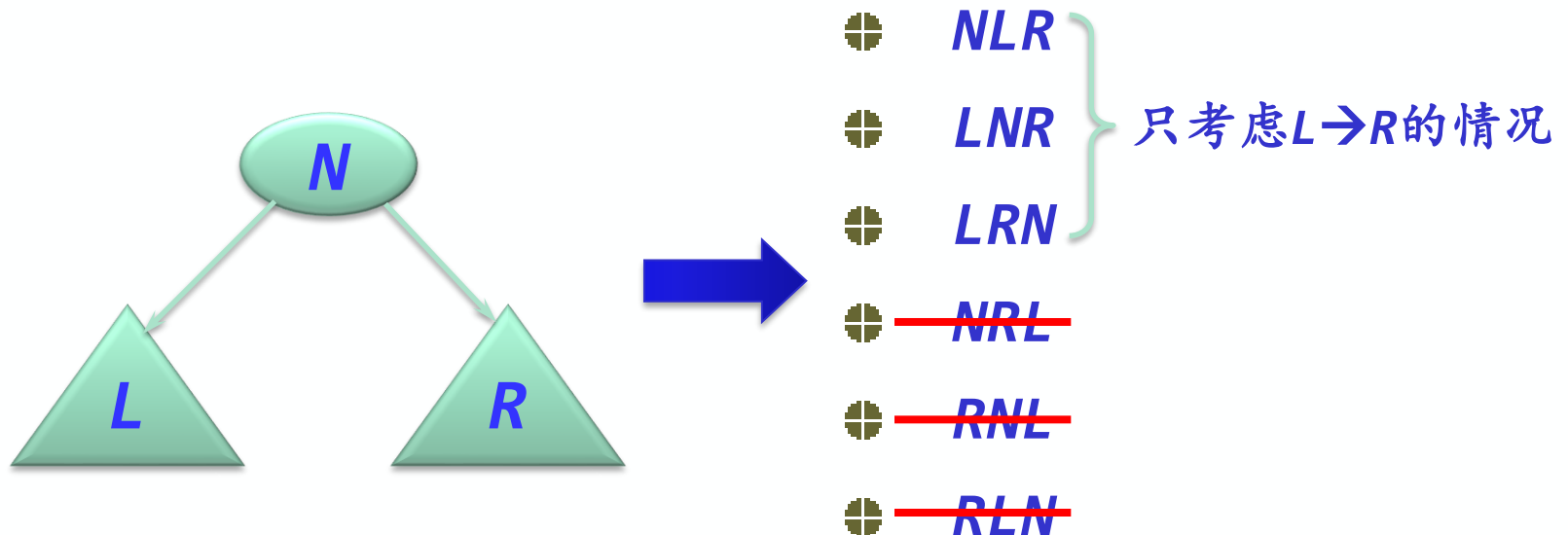
5.7 哈夫曼树及其应用



5.5.1 遍历二叉树 (P147)

遍历定义——指按某条搜索路线遍访每个结点且不重复。

遍历用途——它是树结构插入、删除、修改、查找和排序运算的前提，是二叉树一切运算的基础和核心。



5.5.1 遍历二叉树

主要的遍历方法：

● 先序遍历：

若二叉树非空，则①访问根结点，②先序遍历左子树，③先序遍历右子树。

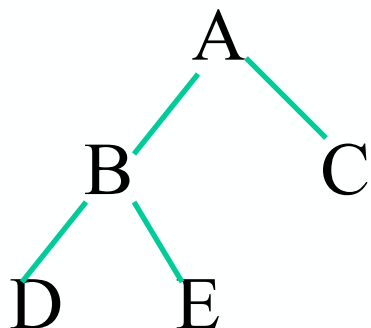
● 中序遍历：

若二叉树非空，则①中序遍历左子树，②访问根结点，③中序遍历右子树。

● 后序遍历：

若二叉树非空，则①后序遍历左子树，②后序遍历右子树，③访问根结点。

注意：先序、中序、后序遍历算法都是递归的。



先序遍历: **A B D E C**

中序遍历: **D B E A C**

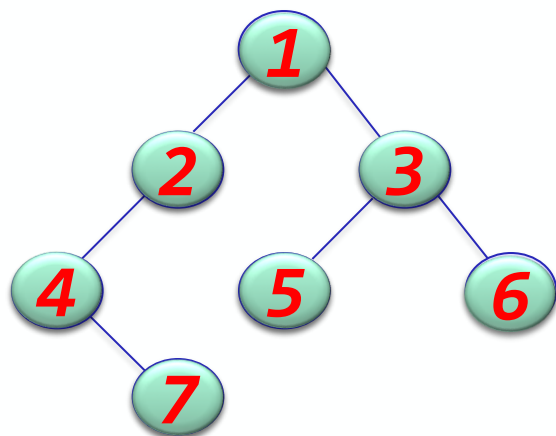
后序遍历: **D E B C A**

口诀:

先序遍历, 即根—左—右

中序遍历, 即左—根—右

后序遍历, 即左—右—根



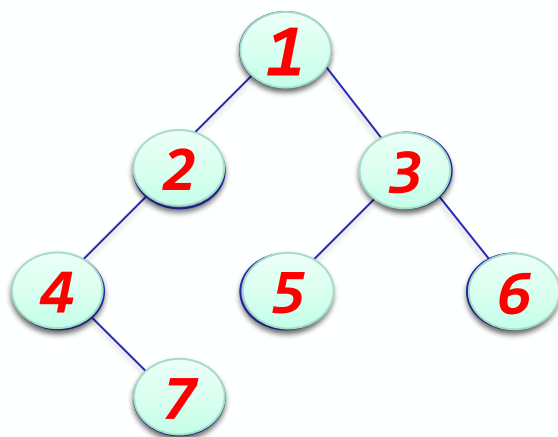
根左右

先序遍历序列:

1 2 4 7 3 5 6

遍历完毕

先序序列的第一个结点是根结点



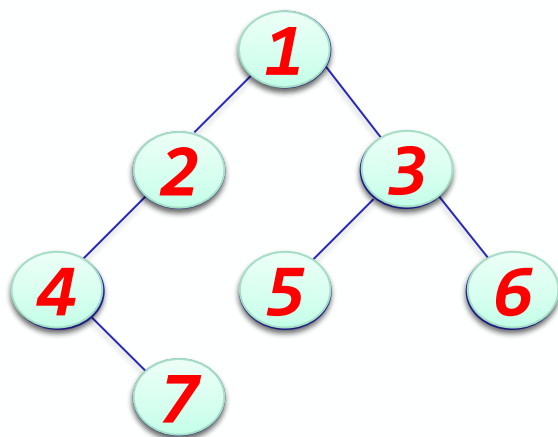
左根右

中序遍历序列:

4 7 2 **1** 5 3 6

遍历完毕

中序序列的根结点左边是左子树的结点，
右边是右子树的结点。



左右根

后序遍历序列:

7 4 2 5 6 3 1

遍历完毕

后序序列的最后一个结点是根结点

【练习11】对二叉树的结点从1开始进行连续编号，要求每个结点的编号大于其左、右孩子的编号。同一结点的左右孩子中，其左孩子的编号小于其右孩子的编号，可采用（ ）遍历实现编号。

A. 先序

B. 中序

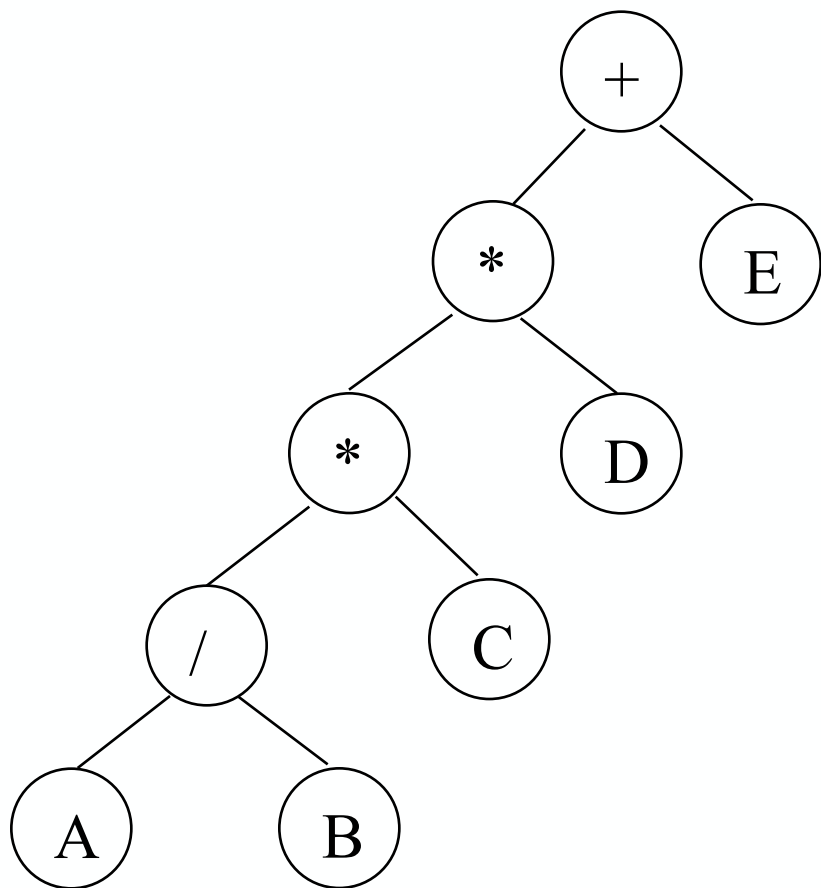
C. 后序

D. 从根结点开始按层次遍历

答案：C

解释：根据题意可知按照先左孩子、再右孩子最后双亲结点的顺序遍历二叉树，即后序遍历二叉树。

用二叉树表示算术表达式(P148类似)



先序遍历

$+ * * / A B C D E$

中序遍历

$A / B * C * D + E$

后序遍历

$A B / C * D * E +$

5.5 遍历算法—以先序遍历为例(P148)

Status PreOrderTraverse(BiTree T)

{

if(T==NULL) return OK; //空二叉树

else

{

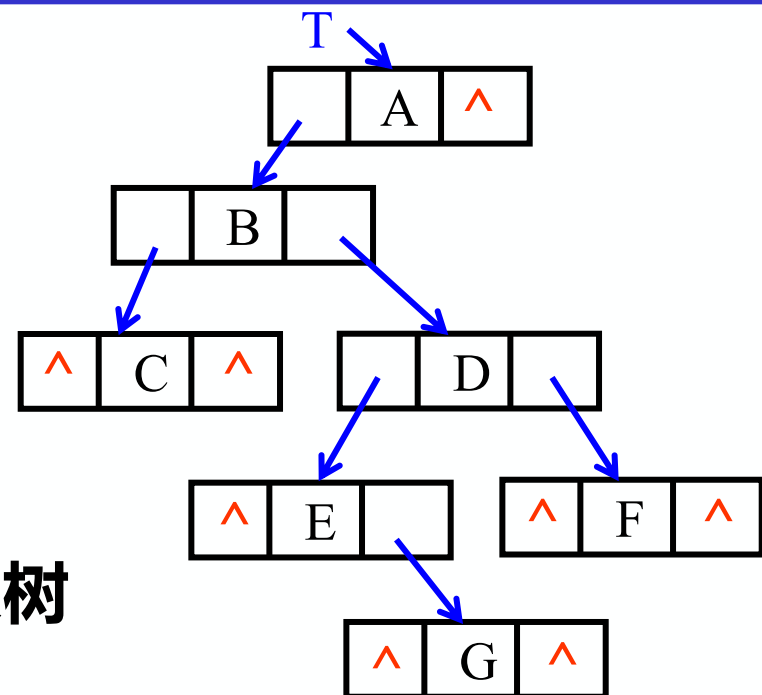
printf("%c", T→data); //访问根结点

PreOrderTraverse(T→lchild); //递归遍历左子树

PreOrderTraverse(T→rchild); //递归遍历右子树

}

}



```

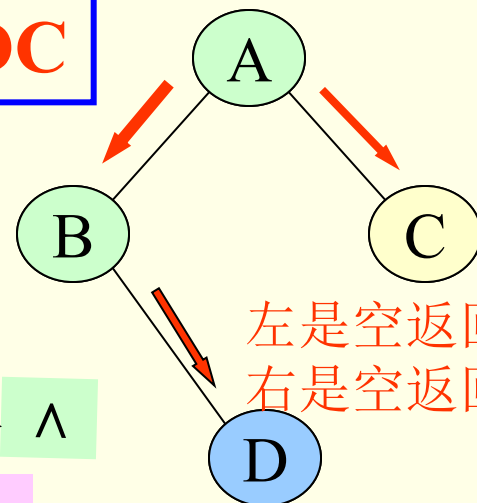
Status PreOrderTraverse(BiTree T){
    if(T==NULL) return OK;
    else{
        printf("%c", T->data);
        PreOrderTraverse(T->lchild);
        PreOrderTraverse(T->rchild); }
}

```

先序序列: **ABDC**

左是空返回

左是空返回
右是空返回



主程序

Pre(T)

T → A
printf(A);
pre(T → L);
pre(T → R);

T → B
printf(B);
pre(T → L);
pre(T → R);

T → C
printf(C);
pre(T → L);
pre(T → R);

T → Λ

返回

T → D

printf(D);
pre(T → L);
pre(T → R);

T → Λ

返回

T → Λ

返回

T → Λ

返回

T → Λ

返回

遍历算法的比较

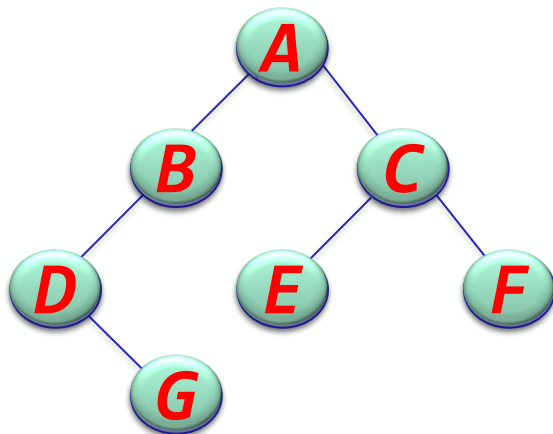
```
Status PreOrderTraverse(BiTree T){  
    if(T==NULL) return OK;  
    else{  
        printf("%c", T→data);  
        PreOrderTraverse(T→lchild);  
        PreOrderTraverse(T→rchild);  
    }  
} // 先序遍历
```

```
Status InOrderTraverse(BiTree T){  
    if(T==NULL) return OK;  
    else{  
        InOrderTraverse(T→lchild);  
        printf("%c", T→data);  
        InOrderTraverse(T→rchild);  
    } } // 中序遍历
```

```
Status PostOrderTraverse(BiTree T)  
{  
    if(T==NULL) return OK;  
    else{  
        PostOrderTraverse(T→lchild);  
        PostOrderTraverse(T→rchild);  
        printf("%c", T→data);  
    } } // 后序遍历
```


● 层序遍历 (P150):

若树非空，从上到下、从左至右按层次进行遍历



层序遍历序列:

A **B** **C** **D** **E** **F** **G** 遍历完毕

无论是哪种遍历方法，对含 n 个结点的二叉树进行遍历，时间复杂度均为 $O(n)$ 。

【练习12】先序为A、B、C，后序为C、B、A的二叉树共有（ ）棵。

A. 5

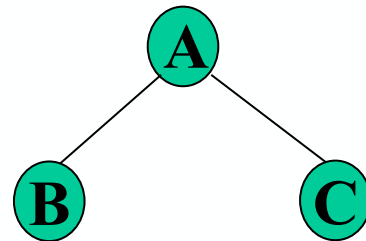
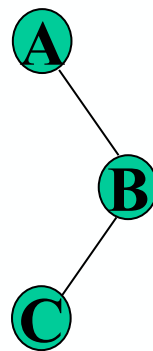
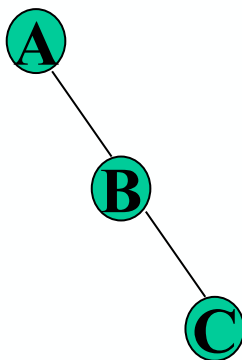
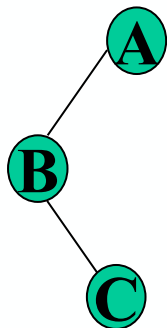
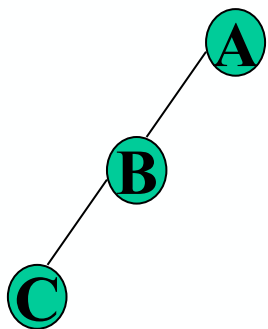
B. 2

C. 3

D. 4

答案：D

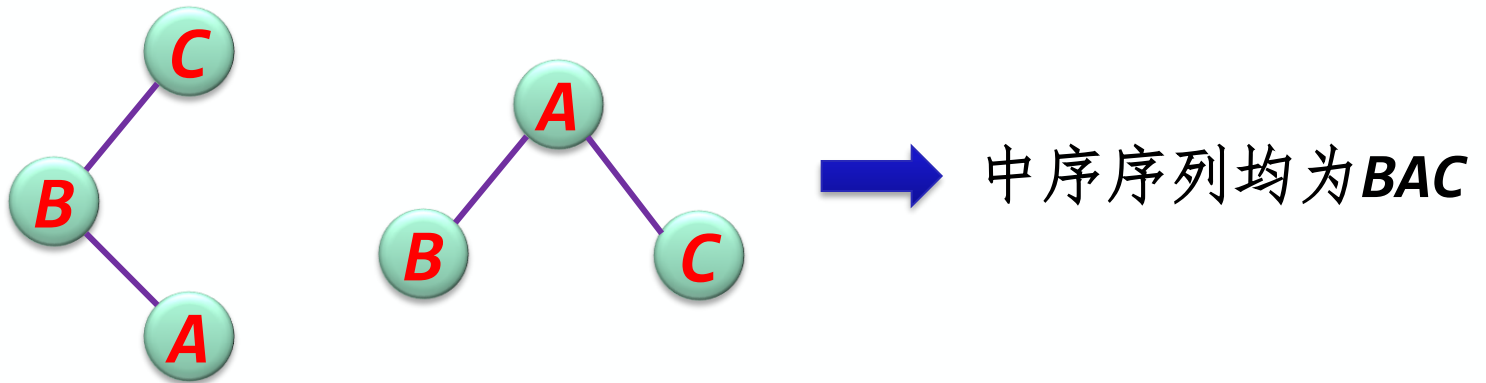
解释：三个结点二叉树有5种，满足要求前四种



如何唯一确定一棵二叉树 (P150-151)

- 一棵二叉树具有**唯一**先序序列、中序序列和后序序列。
- 但**不同的二叉树可能**具有相同的先序序列、中序序列或后序序列。

例如



给定一种遍历序列（先序序列、中序序列或后序序列）
不能唯一确定一棵二叉树。

那么给定两种遍历序列呢？

以下命题成立否？

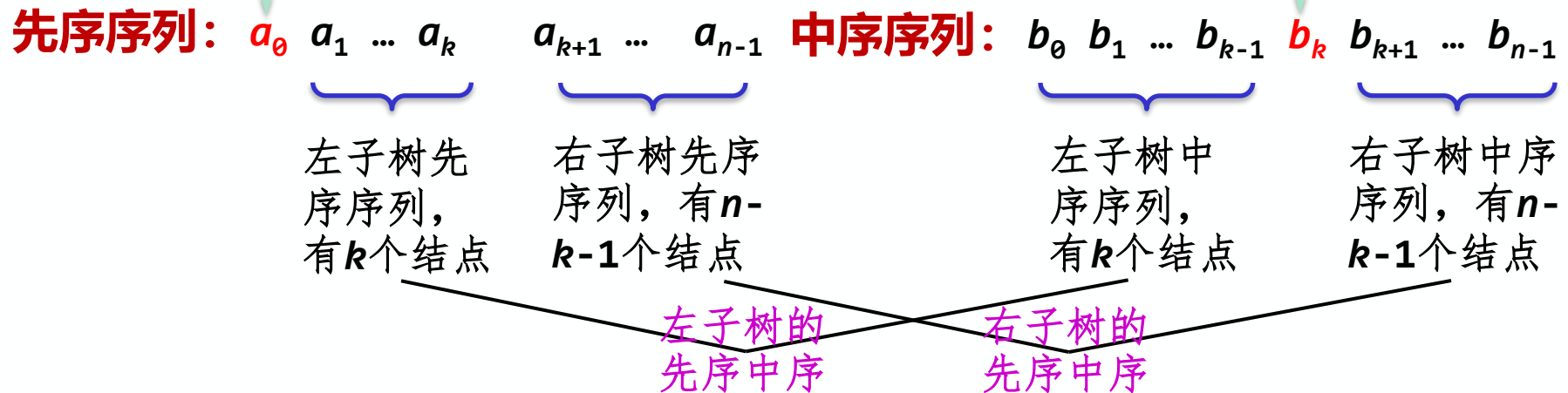
- 同时给定一棵二叉树的先序序列和中序序列能唯一确定这棵二叉树。 ✓
- 同时给定一棵二叉树的中序序列和后序序列能唯一确定这棵二叉树。 ✓
- 同时给定一棵二叉树的先序序列和后序序列能唯一确定这棵二叉树。 ✗

各种遍历序列提供的信息：

- 先序序列 $\longrightarrow$ 根结点
- 中序序列 $\longrightarrow$ 根结点已知时，可以确定左右子树
- 后序序列 $\longrightarrow$ 根结点

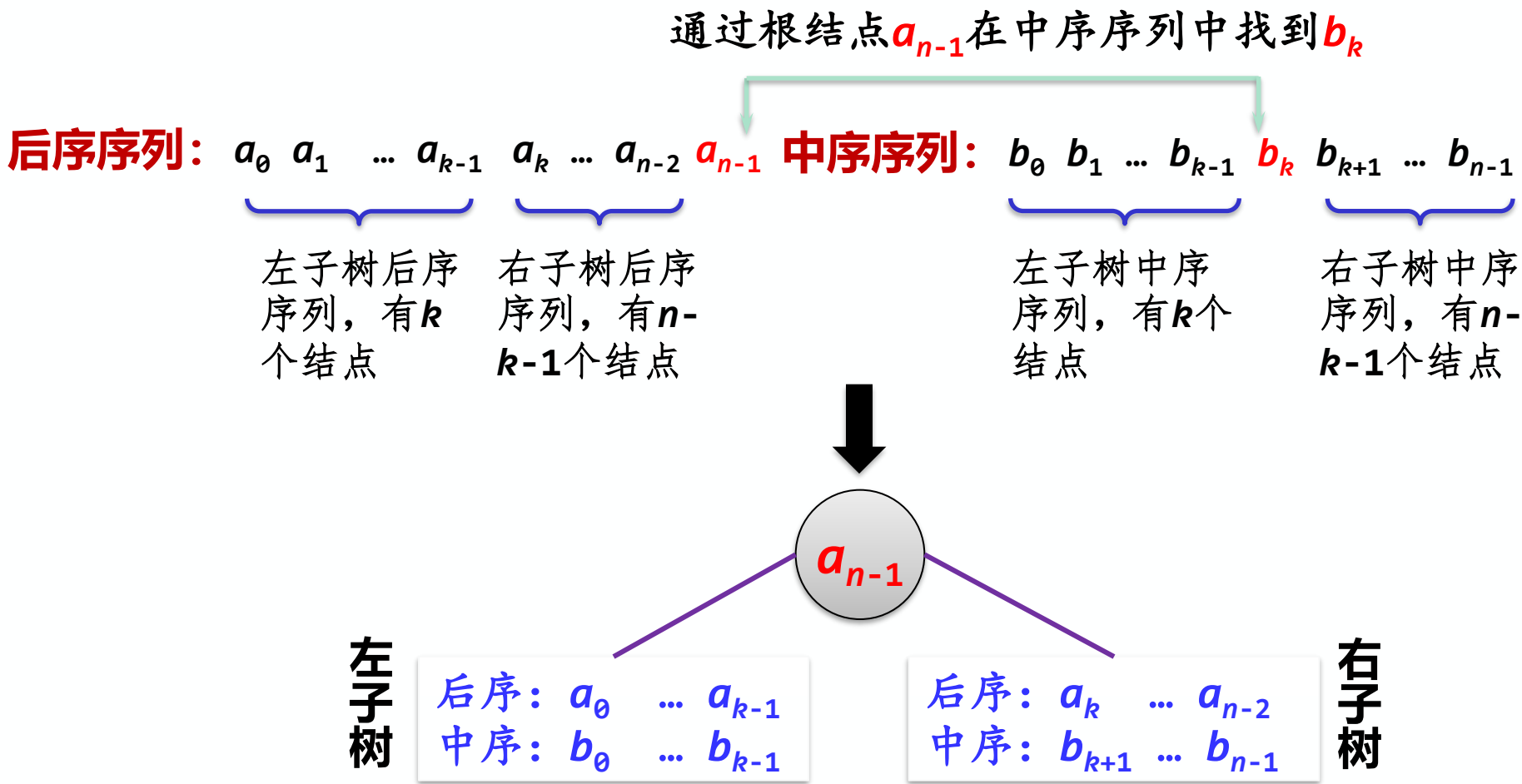
因此，任何 n ($n > 0$) 个不同结点的二叉树，都可由它的先序序列和中序序列唯一地确定。

通过根结点 a_0 在中序序列中找到 b_k



思考

任何 n ($n > 0$) 个不同结点的二叉树，都可由它的中序序列和后序序列唯一地确定。



举例

【例】已知一棵二叉树的**中序序列**和**后序序列**分别是 **BDCEAFHG** 和 **DECBHGFA**，请画出这棵二叉树。

- ①由后序遍历特征（左右根），**根结点必在后序序列尾部（A）**；
- ②由中序遍历特征（左根右），根结点必在中间，而且其左部必全部是左子树结点（**BDCE**），其右部必全部是右子树结点（**FHG**）；
- ③根据后序中的DECB子树可确定B为A的左子树的根结点，根据HGF可确定F为A的右子树的根结点；以此类推。

中序遍历: **B** D C E **A** **F** H G

后序遍历: D E C **B** H G **F** **A**

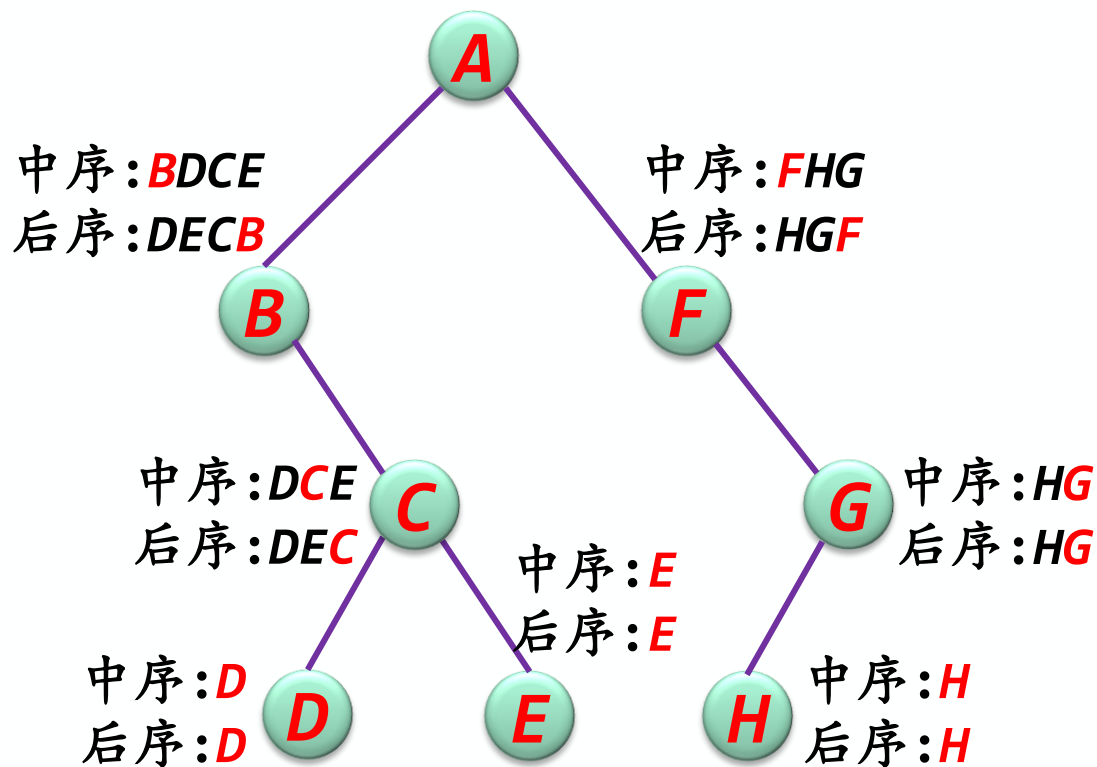
先序序列 → 根结点

中序序列 → 根结点已知时, 可以确定左右子树

后序序列 → 根结点

中序: **B**DCE**A**FHG

后序: DEC**B**HG**F****A**



二叉树构造完毕

练习

已知先序序列为**ABDGCEF**，中序序列为**DGBAECF**，构造该二叉树，并找出该二叉树后序遍历的结果。

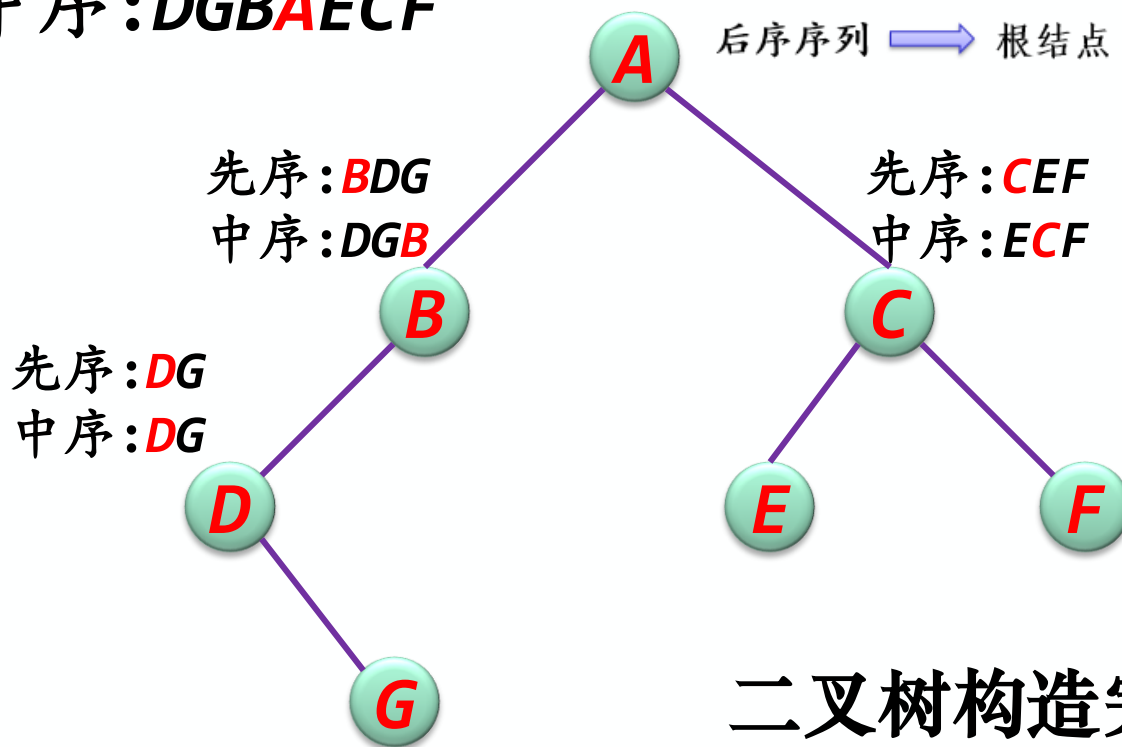
先序：**A**BDGCEF

中序：DGB**A**ECF

先序序列 → 根结点

中序序列 → 根结点已知时，可以确定左右子树

后序序列 → 根结点



二叉树构造完毕

后序遍历的结果： G D B E F C A

5.5.2 线索化二叉树(P154)

(1) 遍历二叉树是按**一定的规则**将树中的结点排列成一个线性序列，得到二叉树中结点的先序、中序和后序序列。

(2) 这实质上是对非线性结构进行线性化操作，使每个结点（除了根结点和最后一个结点）在这些线性序列中有且仅有一个**直接前驱和直接后继**。

比如**中序序列**为DGBAECF，我们知道'B'的前驱是'G'、后继是'A'

比如**先序序列**为ABDEFC，我们知道'E'的前驱是'D'、后继是'F'

(3) 但是，当以**二叉链表作为存储结构**时，只能找到结点的左、右孩子信息，而不能直接得到结点在任一序列中的前驱和后继信息，这种信息只有在遍历的动态过程中才能得到。

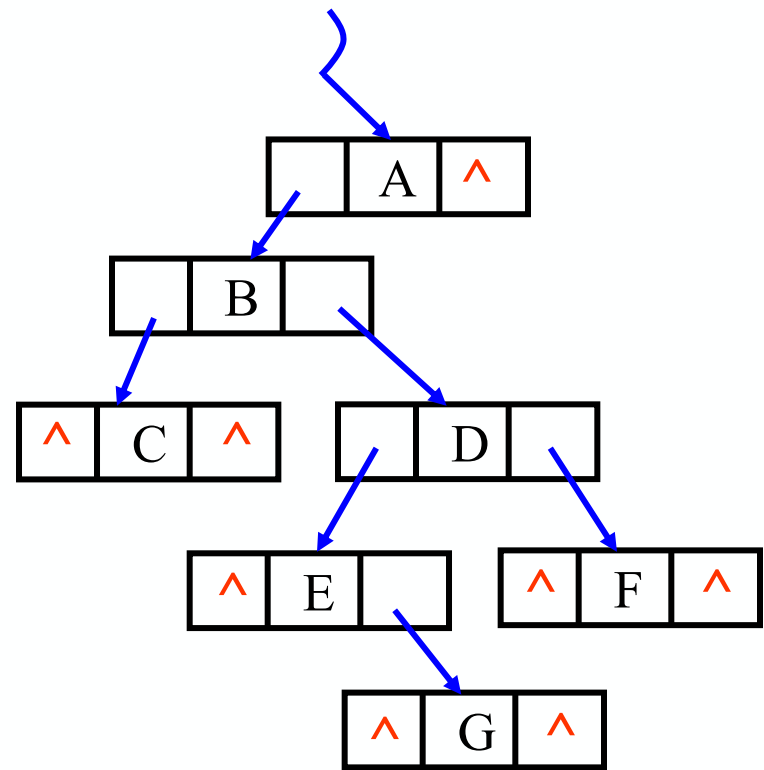
思考

在 n 个结点的二叉链表中，有 $n+1$ 个空指针域

二叉链表空间效率这么低，能否利用这些空闲区存放有用的信息或线索？

——可以利用这些空闲的指针域来存放结点的直接前驱和直接后继信息等线索，以加快查找速度。

线索化二叉树



- n 个结点 $\Rightarrow 2n$ 个指针域
- 分支数为 $n-1$ $\Rightarrow$ 非空指针域有 $n-1$ 个
- 空指针域个数 = $2n - (n-1) = n+1$

5.5.2 线索二叉链表(P154)

做如下规定：

- 1) 若结点有左子树，则lchild指向其左孩子；
否则， lchild指向其直接前驱(即线索)；
- 2) 若结点有右子树，则rchild指向其右孩子；
否则， rchild指向其直接后继(即线索)。

为了避免混淆，增加两个标志域

lchild	LTag	data	RTag	rchild
--------	------	------	------	--------

5.5.2 线索二叉链表(P154)

lchild	RTag	data	RTag	rchild
--------	-------------	------	-------------	--------

$\text{LTag} = \begin{cases} 0: & \text{lchild域指示结点的左孩子} \\ 1: & \text{lchild域指示结点的前驱} \end{cases}$

$\text{RTag} = \begin{cases} 0: & \text{rchild域指示结点的右孩子} \\ 1: & \text{rchild域指示结点的后继} \end{cases}$

【练习13】在 n 个结点的线索二叉树中，线索的数目为（ ）。

A. $n-1$

B. n

C. $n+1$

D. $2n$

答案：C

解释：只有无左右孩子的指针域才会被转化为指向前驱或后继的线索， n 个结点的二叉树中有 $n+1$ 个空指针域，它们都转化为线索，所以线索的数目为 $n+1$

5.5.2 线索化二叉树(P154)

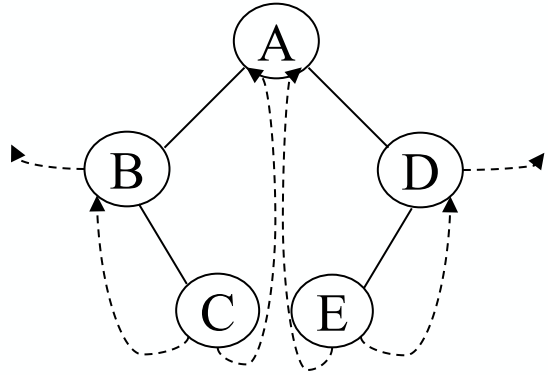
- 采用某种方法遍历二叉树的结果是一个结点的线性序列。
- 修改空链域改为存放指向结点的前驱和后继结点的地址。
- 这样的指向该线性序列中的“前驱”和“后继”的指针，称作**线索**（thread）。
- 创建线索的过程称为**线索化**。
- 线索化的二叉树称为**线索二叉树（逻辑结构）**。
- 对应的存储结构**线索二叉链表**包含五个域：

lchild	ltag	data	rtag	rchild
--------	------	------	------	--------

- 线索二叉树与采用的遍历方法相关，有先序线索二叉树、中序线索二叉树和后序线索二叉树。
- 线索二叉树的目的是为了加快查找结点前驱和后继的速度。

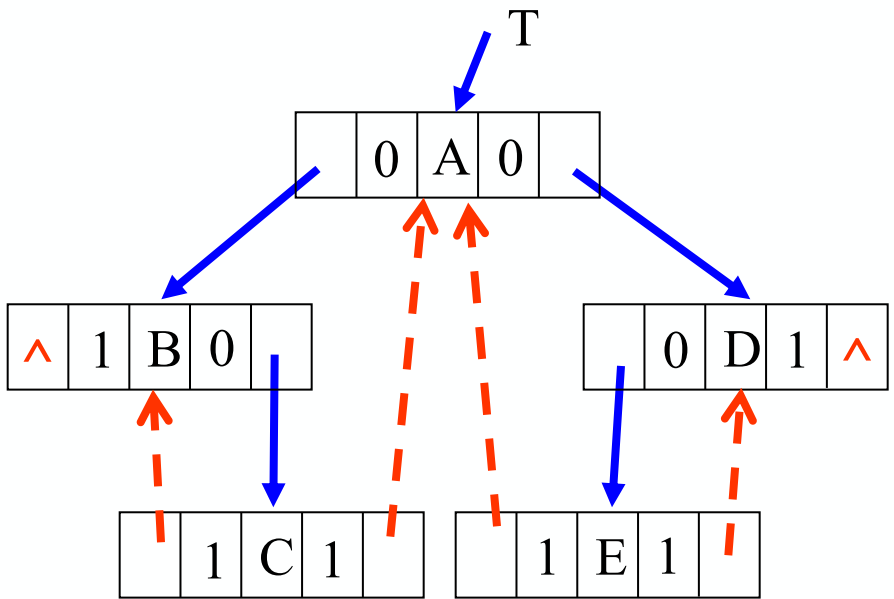
中序线索二叉链表

中序线索二叉树 (逻辑结构)



中序序列：BCAED

lchild	LTAG	data	RTAG	rchild
--------	------	------	------	--------



中序线索链表(存储结构)

- 说明：
- 1.画线索二叉树时，**实线**表示指向其左、右孩子的指针；**虚线**表示指向其直接前驱或直接后继的线索。
 - 2.画线索二叉树，只需要关注那些没有孩子的结点，尤其是叶结点。

【练习14】 若对右图所示二叉树进行中序线索化，则结点x的左、右线索指向的结点分别是（）。

A. e、c

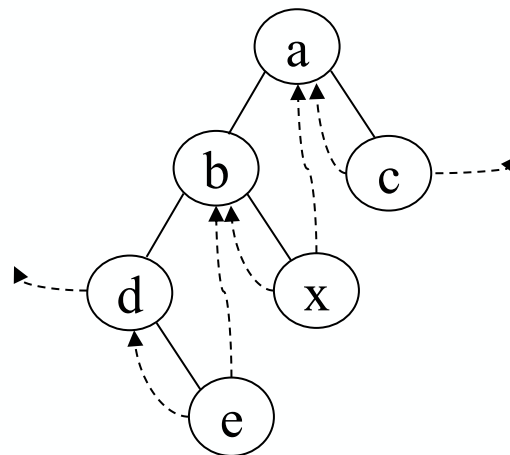
B. e、a

C. d、c

D. b、a

答案：D

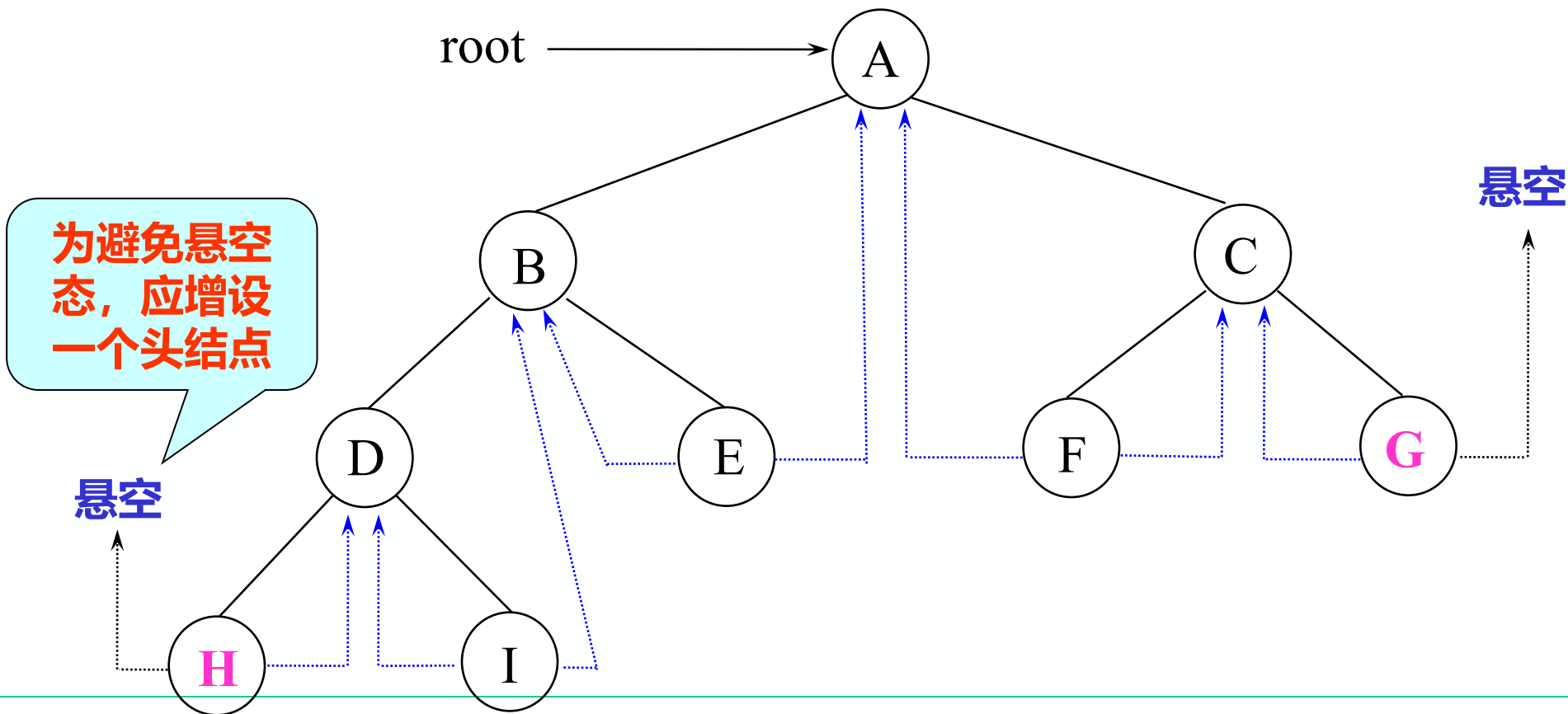
解释：中序序列为debxac



练习

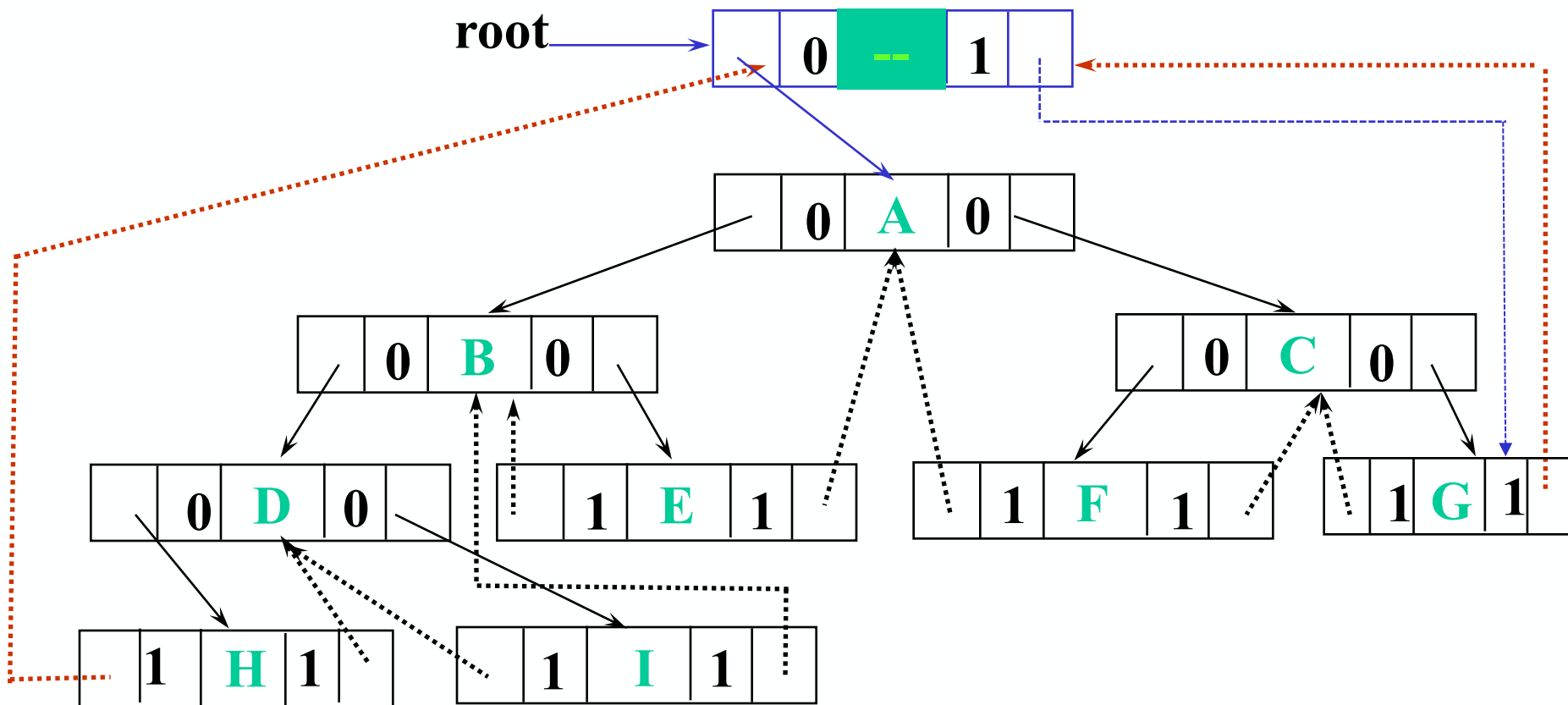
【例】画出以下二叉树对应的**中序线索二叉树**（逻辑结构）

该二叉树中序遍历结果为: **H**, **D**, **I**, **B**, **E**, **A**, **F**, **C**, **G**



对应的存储结构（简称**中序线索链表**）如图所示

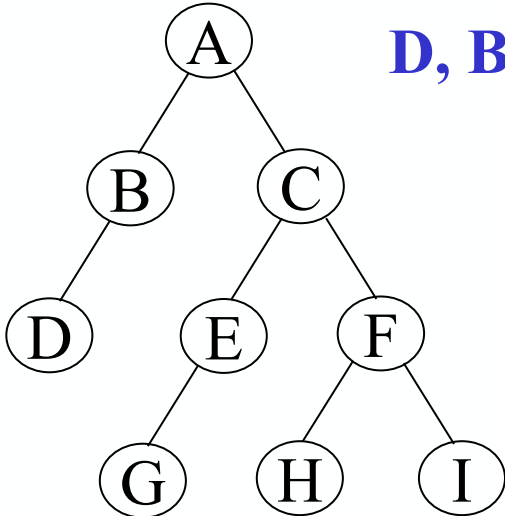
注：此图中序遍历结果为：H, D, I, B, E, A, F, C, G



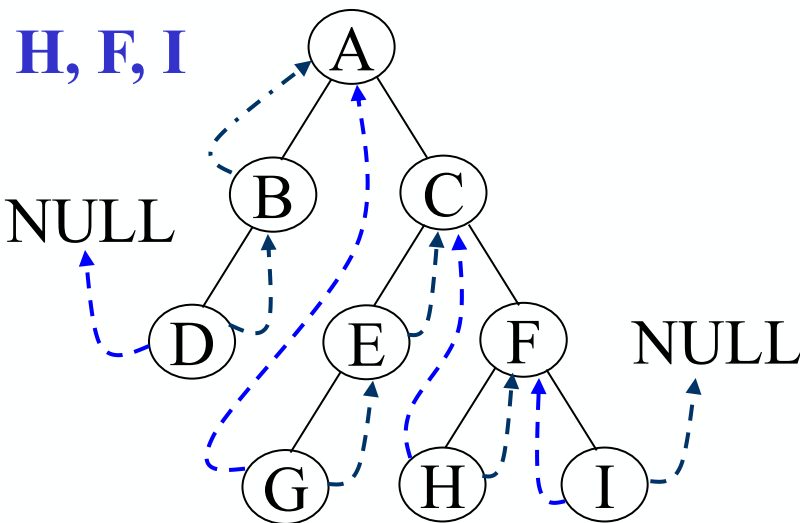
没有前驱后继、悬空的线索都指向头结点

求解下图二叉树对应的中序线索二叉树和中序线索链表

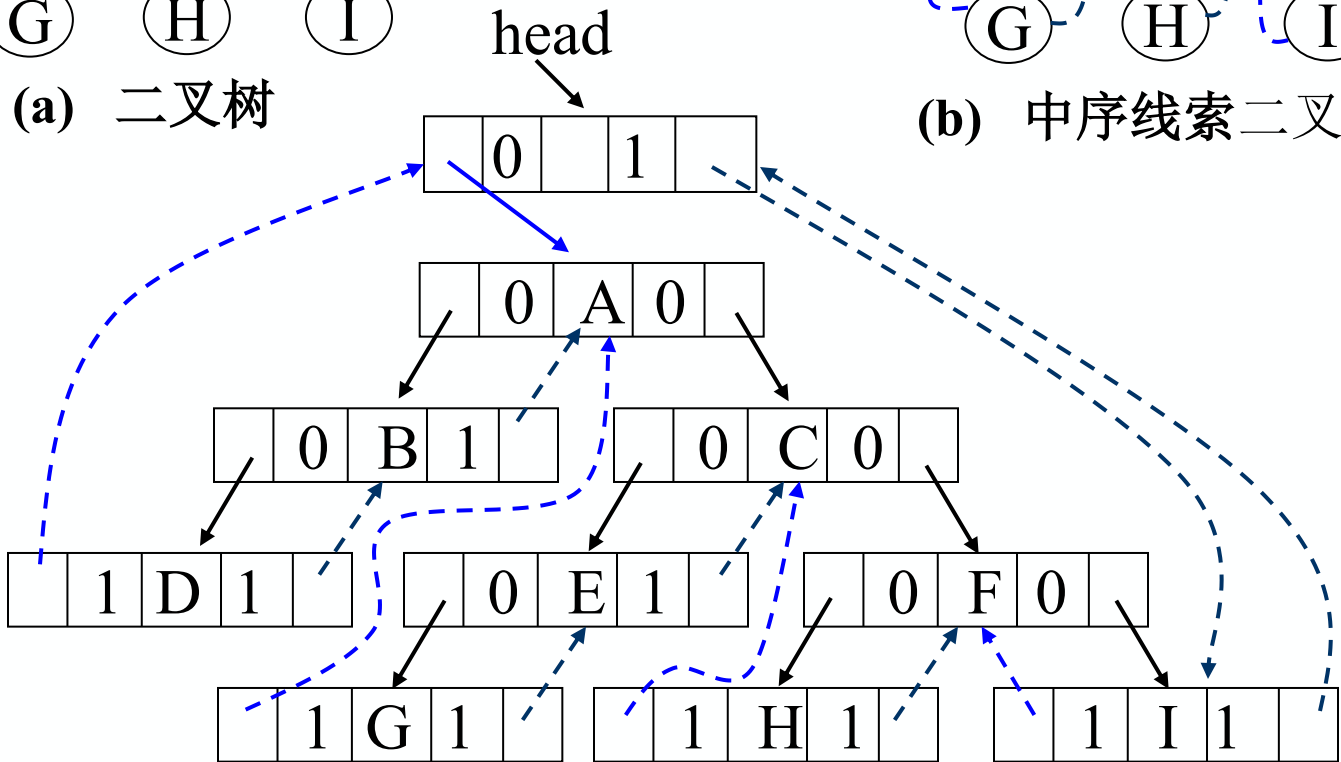
D, B, A, G, E, C, H, F, I



(a) 二叉树



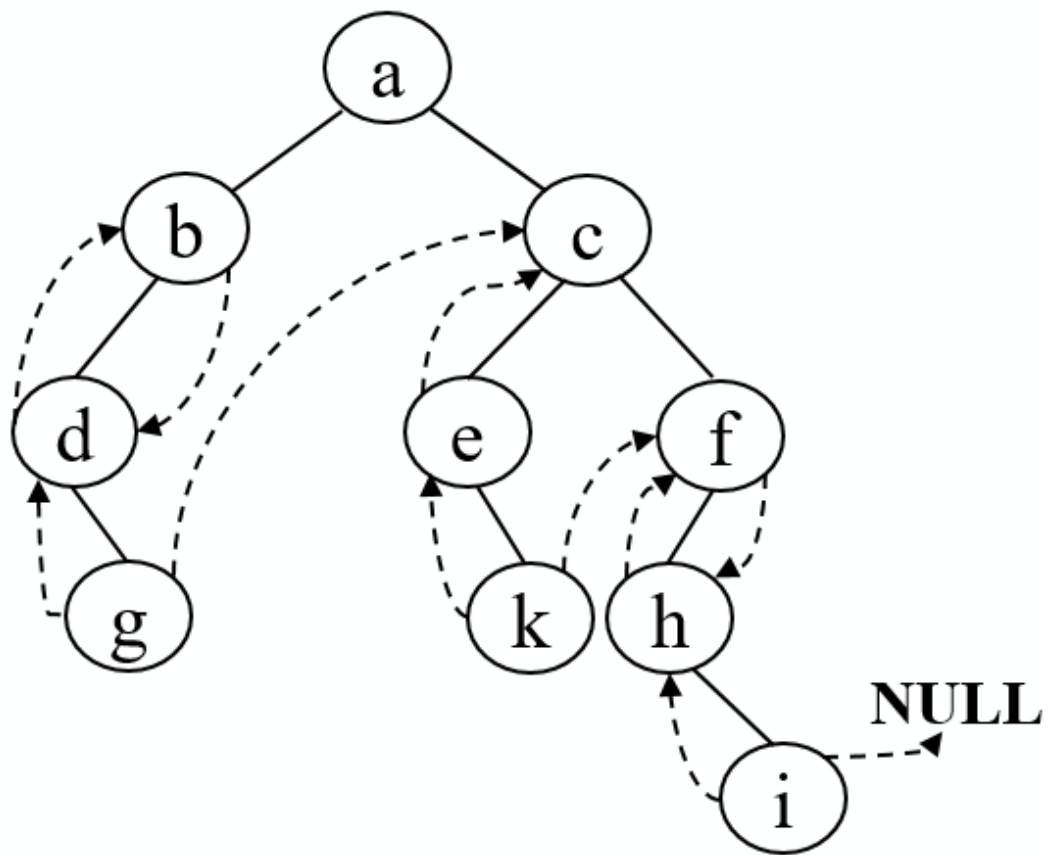
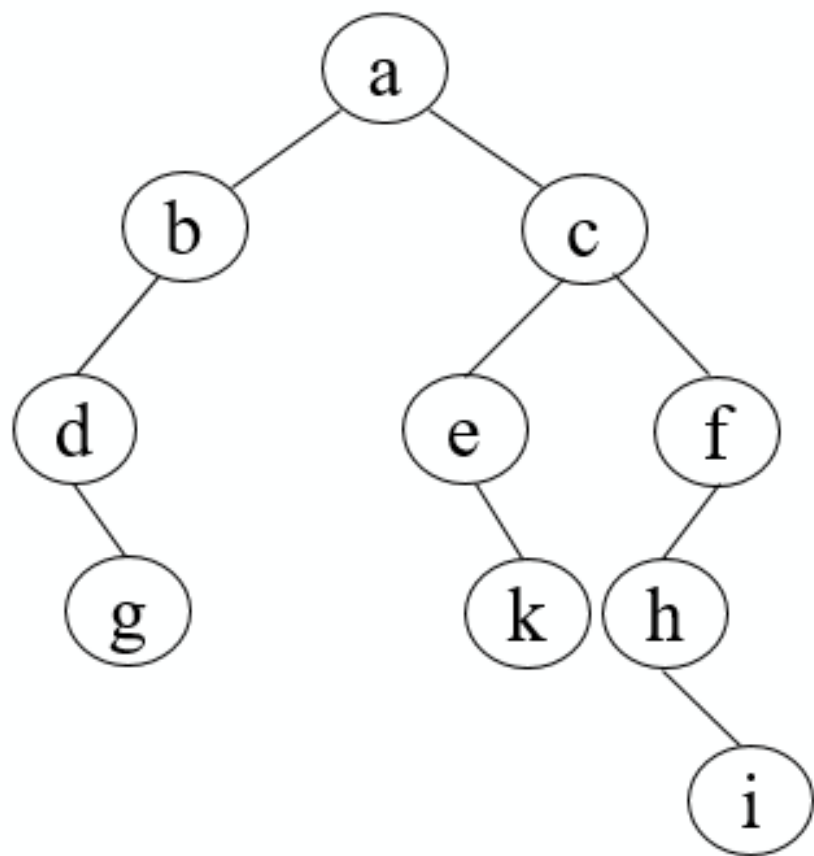
(b) 中序线索二叉树



(c) 中序线索二叉链表

先序线索二叉树

已知一棵二叉树的中序遍历序列和后序遍历序列分别是 **dgbaekchif** 和 **gdbkeihfca**，画出其先序线索二叉树。



先序序列: **a b d g c e k f h i**

第5章 树和二叉树 (P158)

5.1 树的定义和基本术语

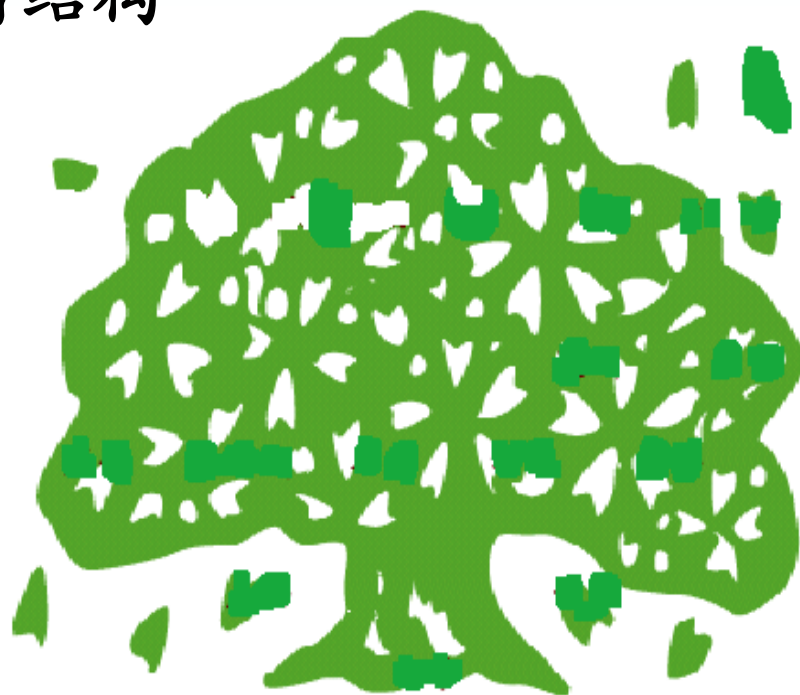
5.3 树的抽象数据类型定义

5.4 二叉树的定义、性质和存储结构

5.5 遍历二叉树和线索二叉树

5.6 树和森林

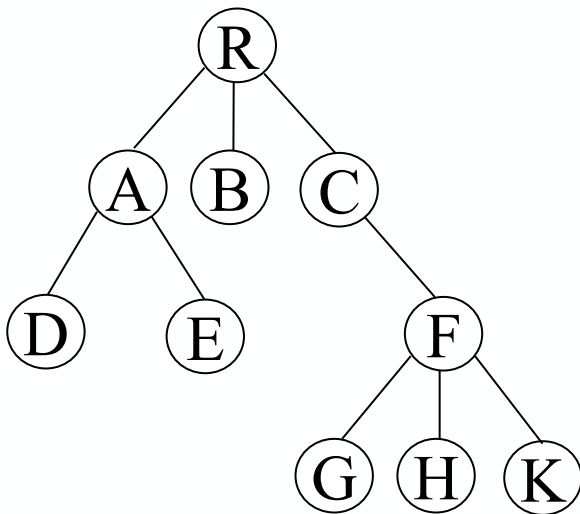
5.7 哈夫曼树及其应用



5.6.1 树的存储结构(P158)

(1) 双亲表示法(顺序存储结构)

用一组连续的存储空间来存储树的结点，每个结点除了数据域 data 外，还附设一个 parent 域用以指示其双亲结点的位置。



思考题：该存储结构的优缺点？

伪指针指示其双亲结点的位置

	data	parent
0	R	-1
1	A	0
2	B	0
3	C	0
4	D	1
5	E	1
6	F	3
7	G	6
8	H	6
9	K	6

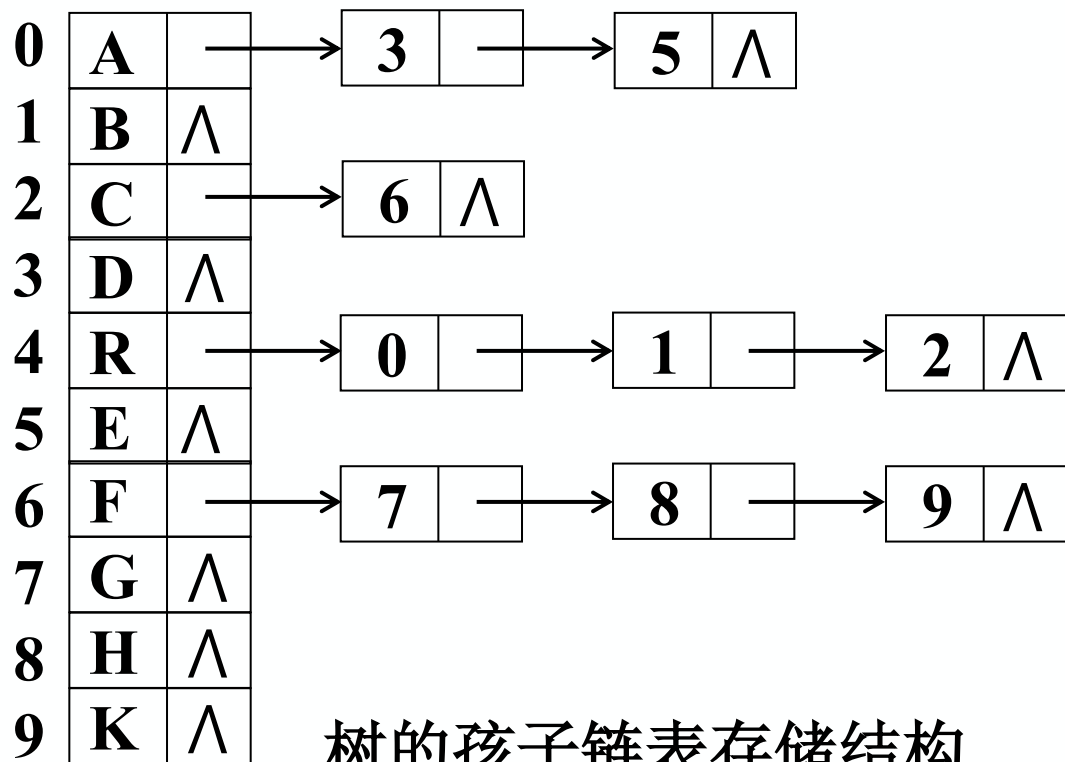
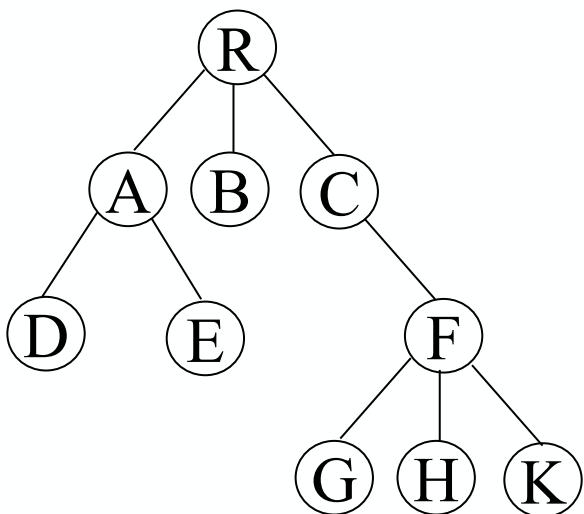
树中任何结点只有唯一的双亲结点

**找双亲容易
找孩子难**

树的双亲存储结构

5.6.1 树的存储结构(P158)

(2) 孩子表示法(链表存储结构)



树的孩子链表存储结构



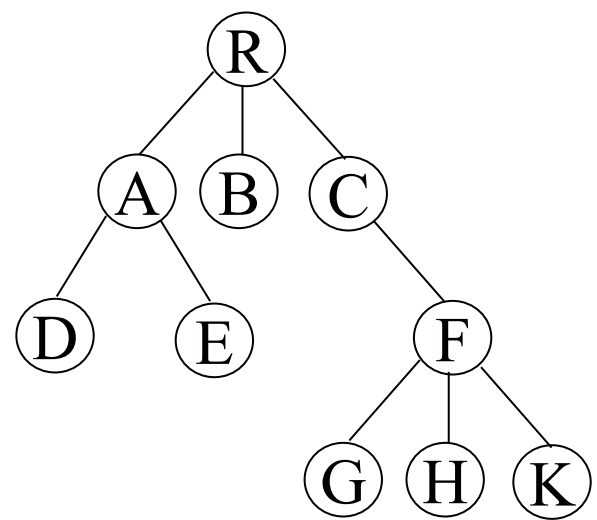
思考题：该存储结构的优缺点？

**找孩子容易
找双亲难**

5.6.1 树的存储结构(P159)

(3) 带双亲的孩子链表

将双亲表示法和孩子表示法结合，即双亲表示和孩子链表合在一起

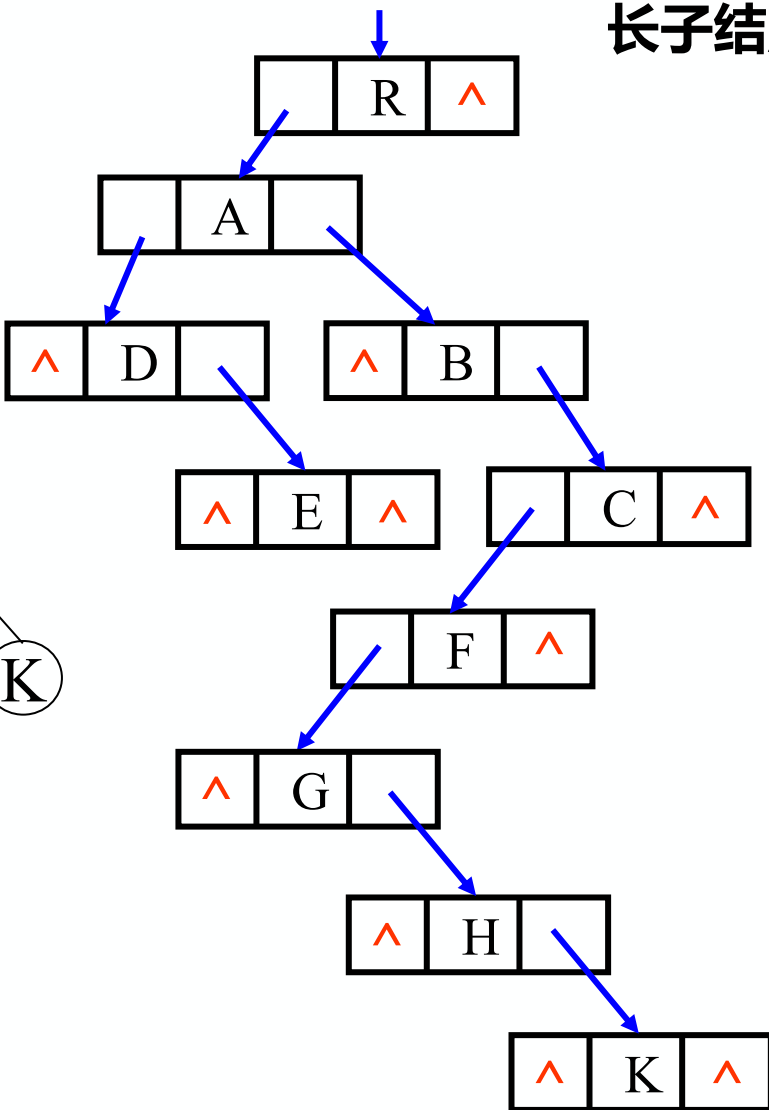
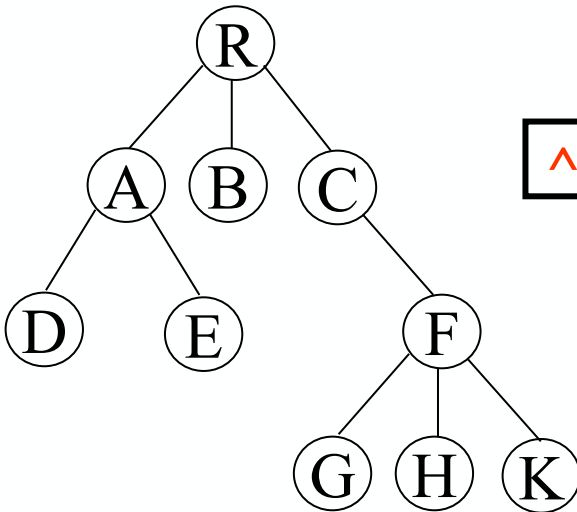


0	4	A	—→	3	—→	5	Λ		
1	4	B	Λ						
2	4	C	—→	6	Λ				
3	0	D	Λ						
4	-1	R	—→	0	—→	1	—→	2	Λ
5	0	E	Λ						
6	2	F	—→	7	—→	8	—→	9	Λ
7	6	G	Λ						
8	6	H	Λ						
9	6	K	Λ						

找双亲容易
找孩子容易

5.6.1 树的存储结构(P159)

(3) 孩子兄弟表示法 (二叉链表表示法)



firstchild	data	nextsibling
------------	------	-------------

↑
长子结点

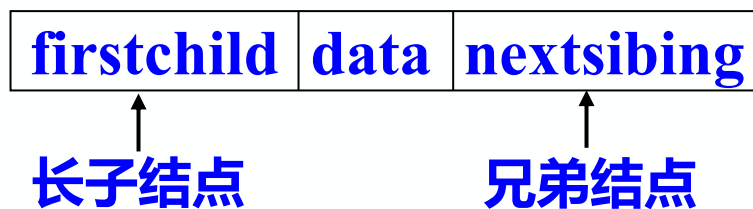
↑
兄弟结点

结点结构

找孩子容易
顺着右指针域

二叉链表表示法

① 树的孩子兄弟表示法



结点结构

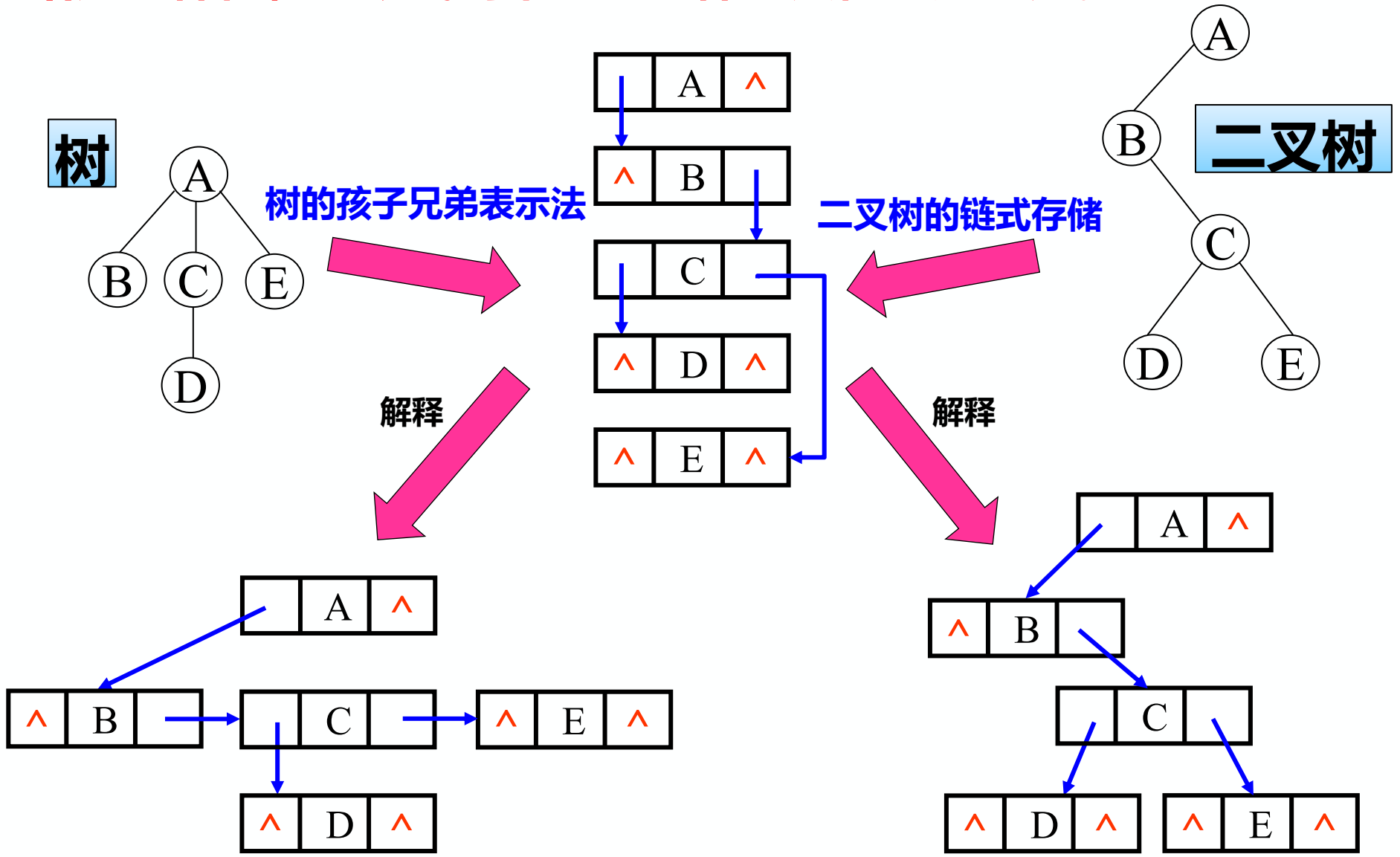
② 二叉树的链式存储



结点结构

5.6.2 森林与二叉树的转换(P160)

给定一棵树，可以找到唯一的一棵二叉树与之对应。



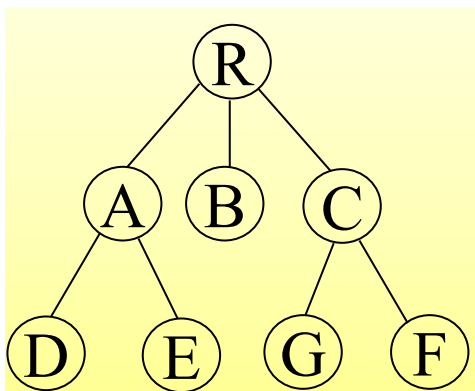
5.6.2 森林与二叉树的转换(P160)

(1) 树转换成二叉树

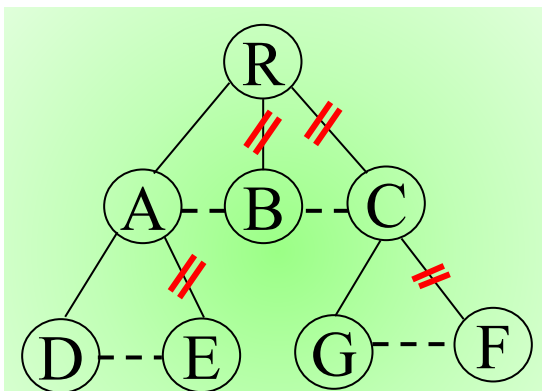
①加虚线：树的每层按从左至右的顺序在兄弟结点之间加虚线相连。

②去连线：除最左的第一个孩子结点外，双亲结点与所有其它孩子结点的连线都去掉。旋转：将树顺时针旋转 45° ，原有的实线左斜。

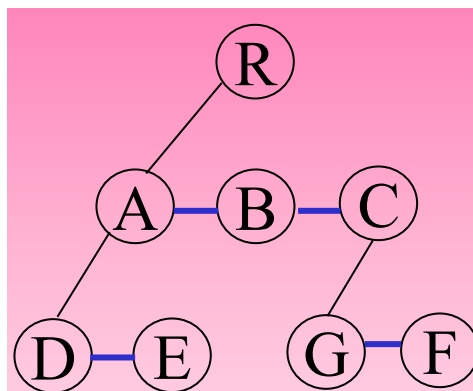
③整型：将旋转后树中的所有虚线改为实线，并向右斜。



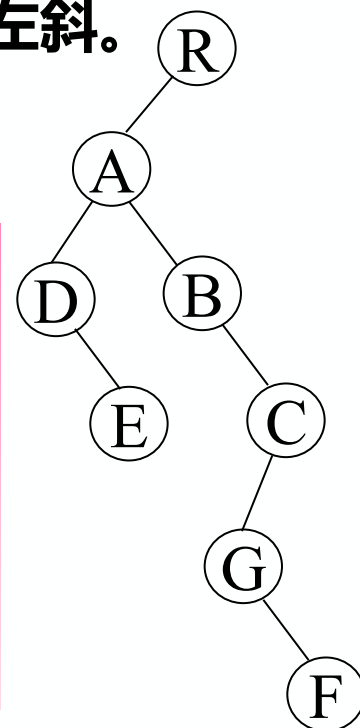
(a) 一般的树



(b) 加虚线，去连线后



树向二叉树的转换过程

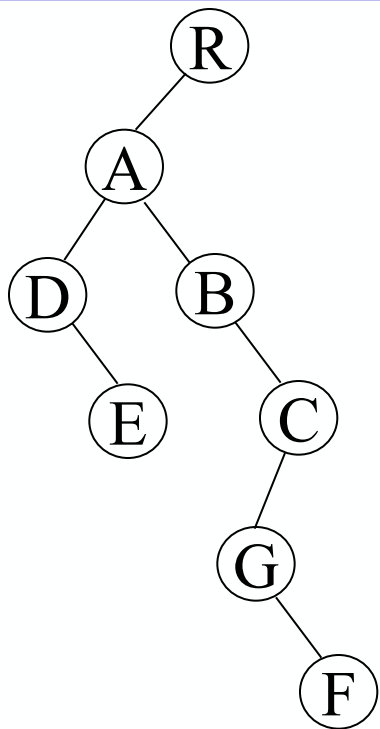


(c) 转换后的二叉树

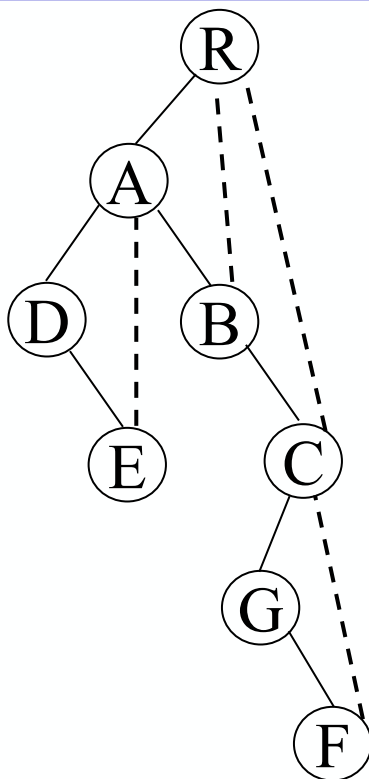
“兄弟相连留长子”

(2) 二叉树转换为树

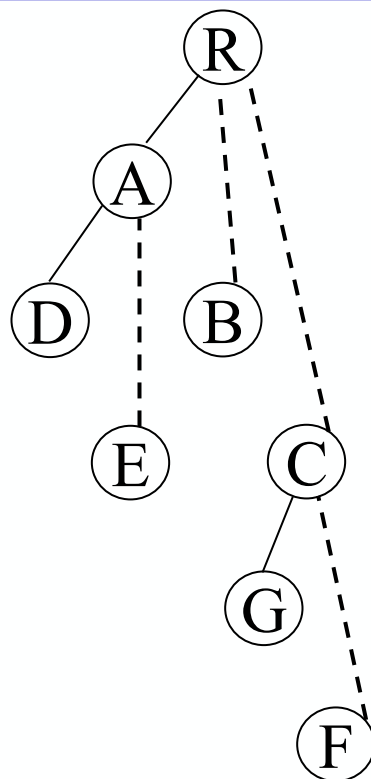
“左孩以右连双亲，去掉之前右孩线”



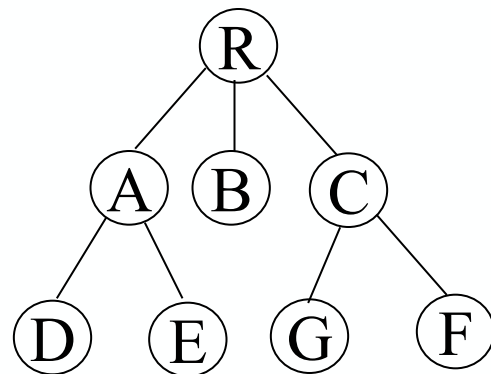
原二叉树



(a) 加虚线后



(b) 去连线后



(c) 还原后的树

①加虚线：若某结点i是其双亲结点的左子树的根结点，则将该结点i的右孩子结点以及沿右孩子链不断地搜索所有的右孩子结点，将所有这些右孩子结点与i结点的双亲结点之间加虚线相连；

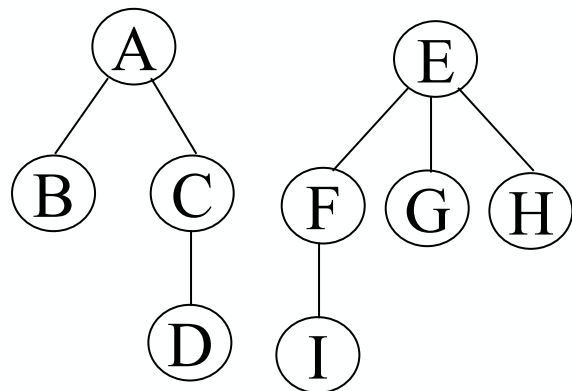
②去连线：去掉二叉树中所有双亲结点与其右孩子结点之间的连线；

③规整化：将图中各结点按层次排列且将所有的虚线变成实线。

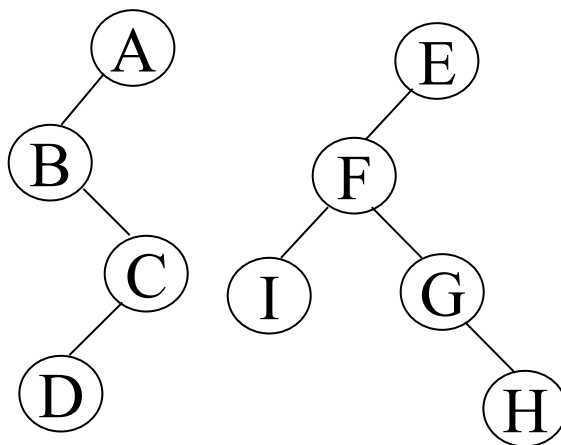
(3) 森林转换成二叉树

- ① 将 $F=\{T_1, T_2, \dots, T_n\}$ 中的每棵树转换成二叉树。
- ② 按给出的森林中树的次序，从最后一棵二叉树开始，每棵二叉树作为前一棵二叉树的根结点的右子树。

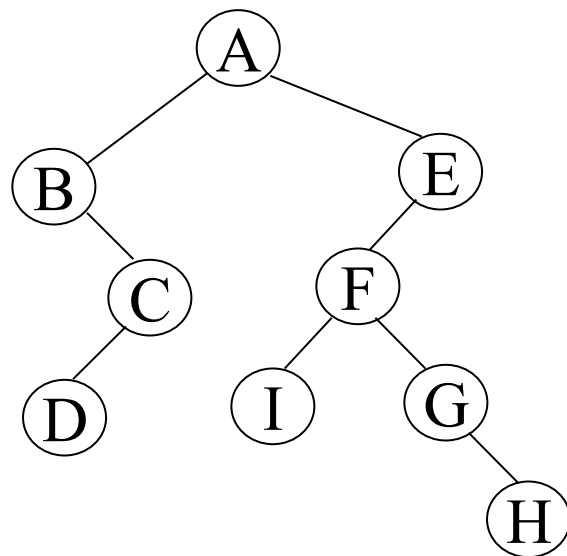
兄弟相连留长子



(a) 森林



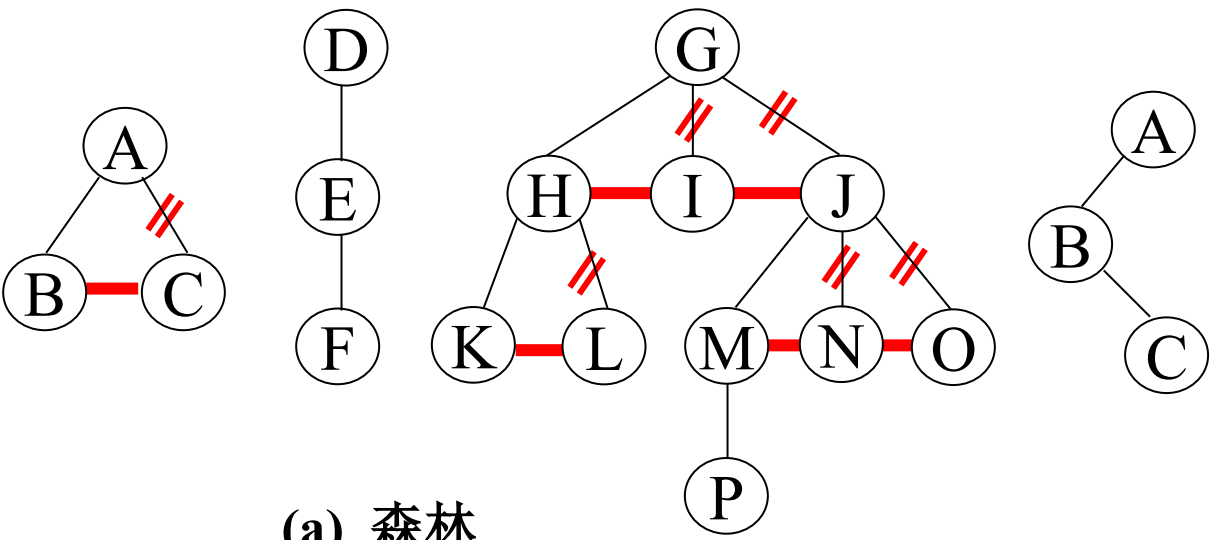
(b) 森林中每棵树
对应的二叉树



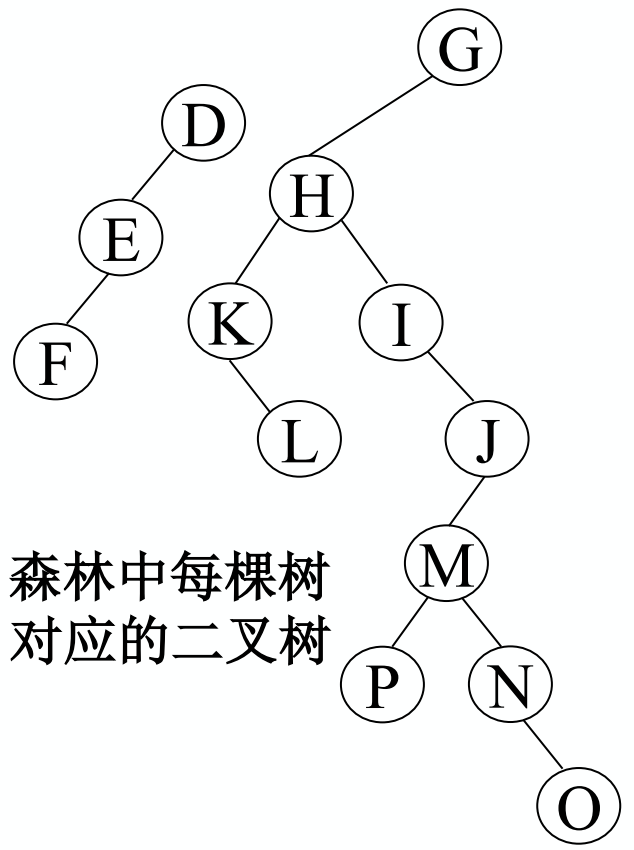
(c) 森林对应的二叉树

森林转换成二叉树的过程

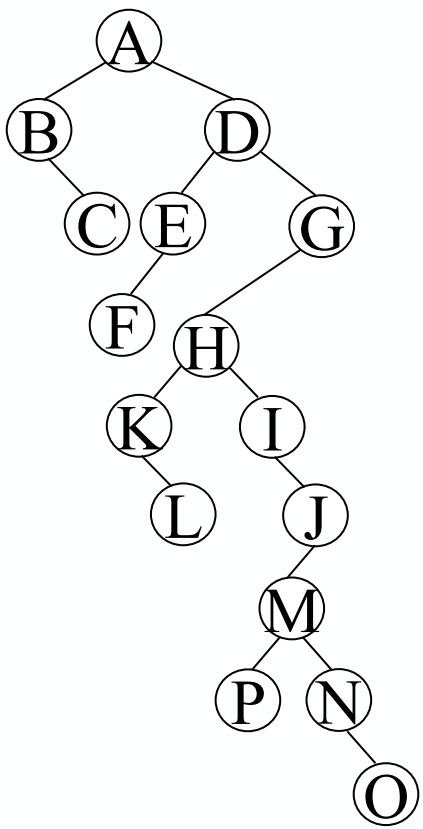
“树变二叉根相连”



(a) 森林



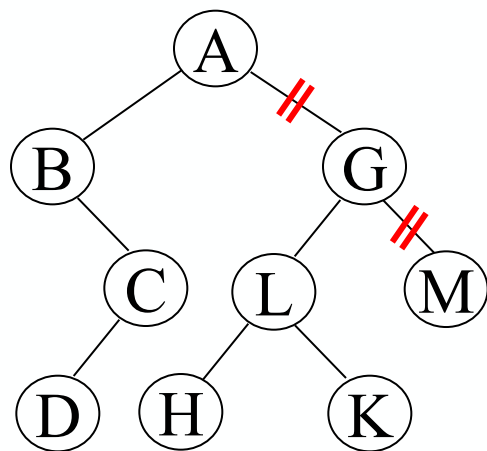
(b) 森林中每棵树对应的二叉树



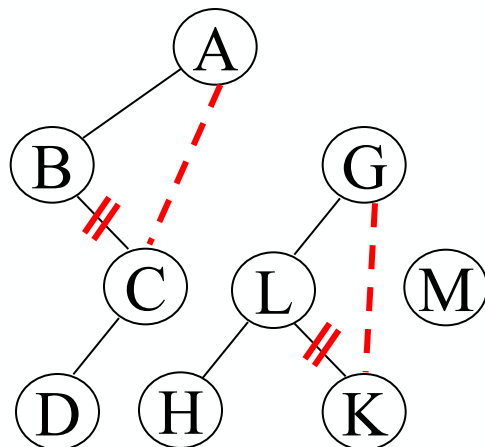
(4) 二叉树转换成森林

- ① **去连线**：将二叉树的根结点与其右子结点以及沿右子结点链方向的所有右子结点的连线全部去掉，得到若干棵孤立的二叉树。
- ② **二叉树的还原**：将各棵孤立的二叉树按还原方法还原成一般的树。

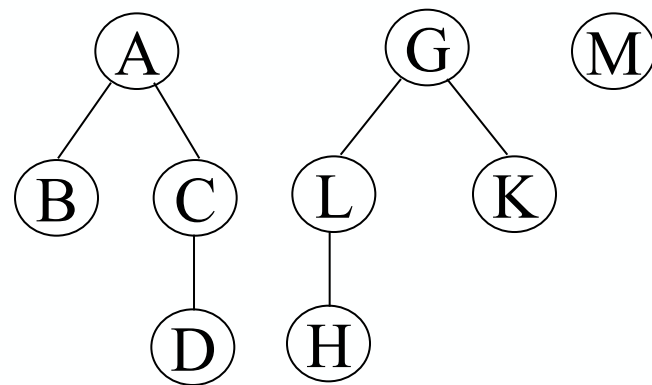
“左孩以右连双亲，去掉之前右孩线”



(a) 二叉树



(b) 去连线后



(c) 还原成森林

二叉树还原成森林的过程

“去掉全部右孩线，孤立二叉再还原”

5.6.3 树和森林的遍历(P161)

树的遍历是指按某种方式访问树中的每个结点且每个结点只被访问一次。树的子树不区分左右，所以树的遍历只有先根和后根。

主要的遍历方法：

● 先根遍历：

若树不空，则先访问根结点，然后依次先根遍历各棵子树。

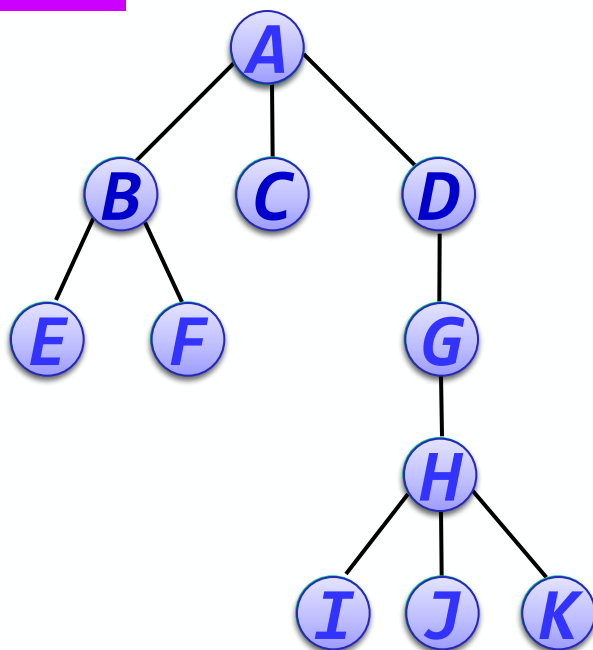
● 后根遍历：

若树不空，则先依次后根遍历各棵子树，然后访问根结点。

注意：先根和后根遍历算法都是递归的。
同一层结点遵循先左后右。

5.6.3 树和森林的遍历(P161)

树的先根遍历示例



若树不空，则先访问根结点，
然后依次先根遍历各棵子树。

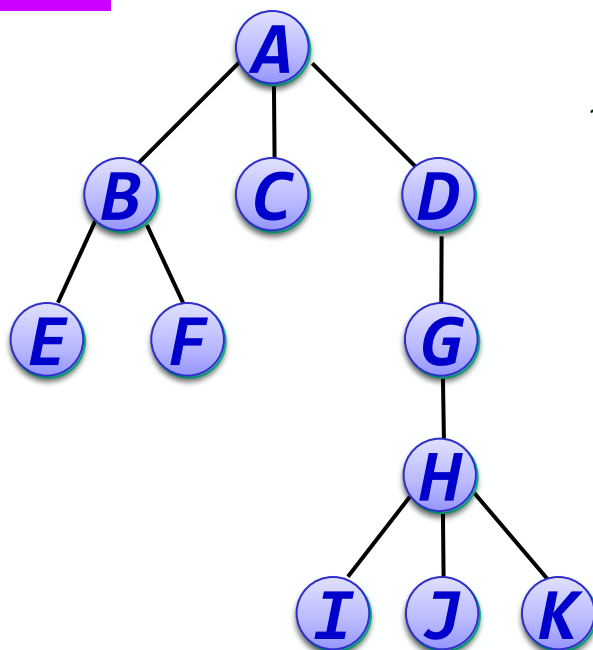
先根遍历的结点访问次序：

A B E F C D G H I J K

遍历完毕

5.6.3 树和森林的遍历(P161)

树的后根遍历示例



若树不空，则先依次后根遍历各棵子树，然后访问根结点。

后根遍历的结点访问次序：

E F B C I J K H G D A

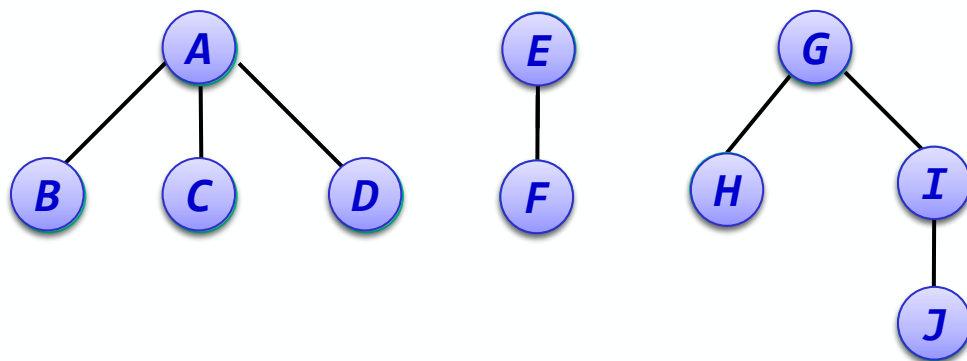
遍历完毕

5.6.3 树和森林的遍历(P161)

森林的遍历示例

设 $F=\{T_1, T_2, \dots, T_n\}$ 是森林，对 F 的遍历有两种方法。

- (1) 先序遍历：按先根遍历树的方式依次遍历 F 中的每棵树。
- (2) 中序遍历：按后根遍历树的方式依次遍历 F 中的每棵树。



先序: **A B C D E F G H I J**

中序: **B C D A F E H J I G**

第5章 树和二叉树 (P162)

5.1 树的定义和基本术语

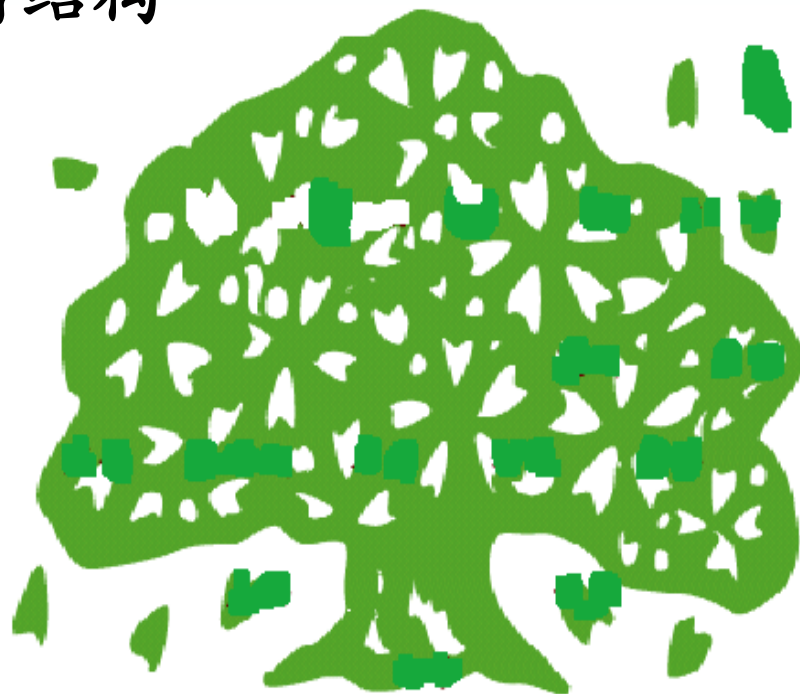
5.3 树的抽象数据类型定义

5.4 二叉树的定义、性质和存储结构

5.5 遍历二叉树和线索二叉树

5.6 树和森林

5.7 哈夫曼树及其应用



5.7.1 哈夫曼树的基本概念(P162)

路 径： 从树中一个结点到另一个结点的之间的分支。

如 $a \rightarrow e$ 路径 abe

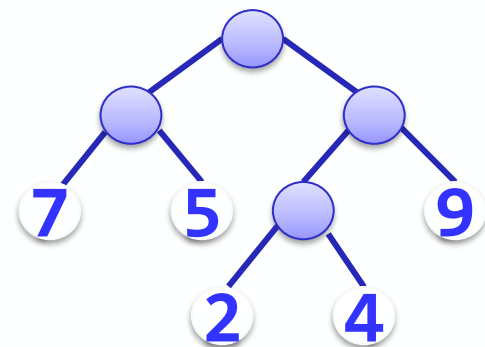
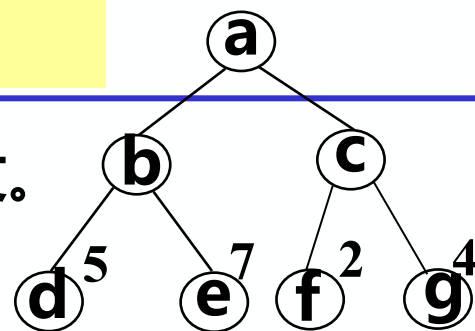
路径长度： 路径上的分支数目 $a \rightarrow e$ 的路径长度 = 2

权： 将树中结点赋予一个有着某种含义的数值，该数值称为结点的权

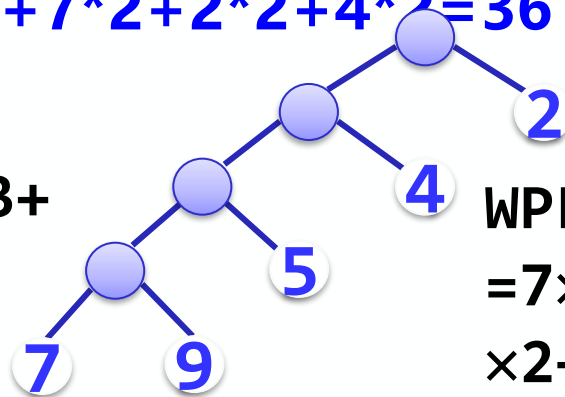
结点带权路径长度： 从该结点到根结点之间的路径长度与结点的权(值)的乘积。如结点e带权路径长度 = $2 * 7 = 14$

树的带权路径长度： 树中所有叶子结点的带权路径长度之和。 $WPL = \sum_{i=1}^{n_0} w_i l_i$

$$WPL = 5 * 2 + 7 * 2 + 2 * 2 + 4 * 2 = 36$$



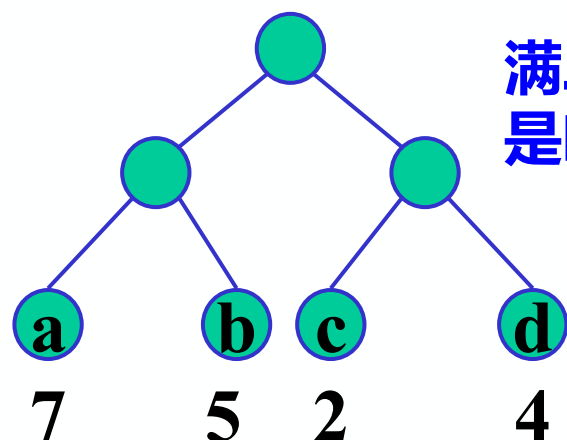
$$\begin{aligned} WPL(T_1) &= 7 \times 2 + 5 \times 2 + 2 \times 3 + \\ &\quad 4 \times 3 + 9 \times 2 \\ &= 60 \end{aligned}$$



$$\begin{aligned} WPL(T_2) &= 7 \times 4 + 9 \times 4 + 5 \times 3 + 4 \\ &\quad \times 2 + 2 \times 1 = 89 \end{aligned}$$

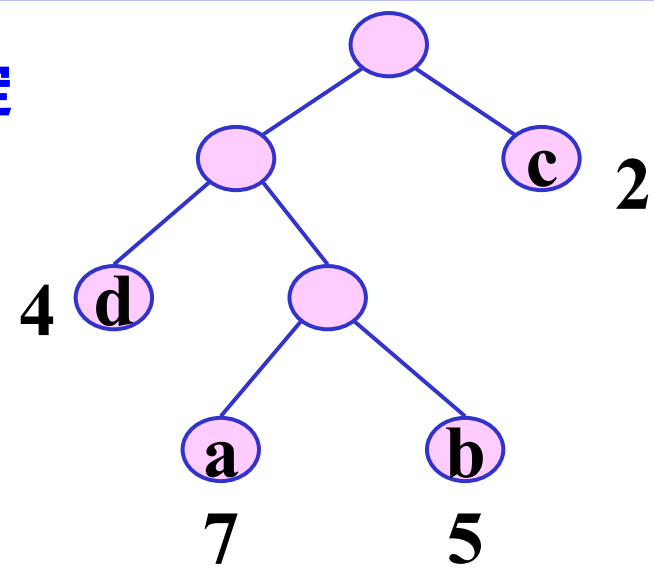
哈夫曼树 (Huffman)： 具有n个叶子结点(每个结点的权值为 w_i)的二叉树不止一棵，但在所有的这些二叉树中，必定存在一棵WPL值最小的树，称这棵树为哈夫曼树(或称最优二叉树)。

权值分别为7, 5, 2, 4, 构造有4个叶子结点的二叉树

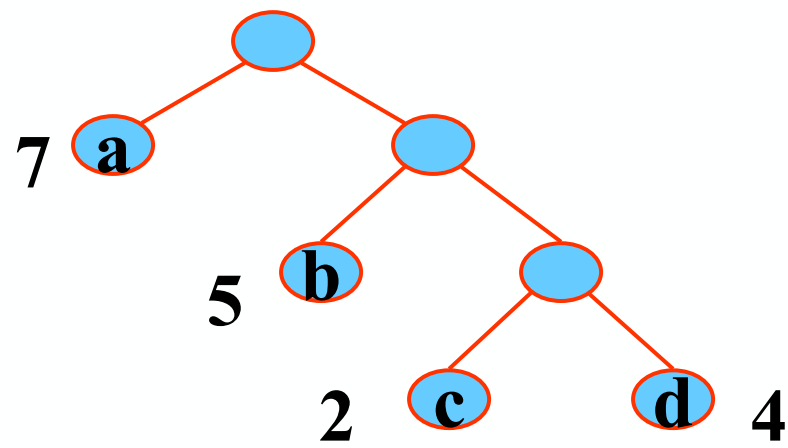


满二叉树不一定是哈夫曼树

$$WPL = 7*2 + 5*2 + 2*2 + 4*2 = 36$$



$$WPL = 7*3 + 5*3 + 2*1 + 4*2 = 46$$



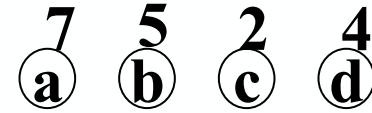
$$WPL = 7*1 + 5*2 + 2*3 + 4*3 = 35$$

构造哈夫曼树的原则：

- 权值越大的叶结点越靠近根结点。
- 权值越小的叶结点越远离根结点。

哈夫曼树的构造过程(P163) 权值分别为7, 5, 2, 4

构造哈夫曼树的**过程**:



(1) 给定的 n 个权值 $\{w_1, w_2, \dots, w_n\}$, 构造 n 棵只有一个带权为 w_i 的根结点的二叉树, 其左右子树均为空, 从而得到一个二叉树的集合 $F=\{T_1, T_2, \dots, T_n\}$ 。

(2) 在 F 中选取**根结点**的权值最小和次小的两棵树作为左、右子树构造一棵新的二叉树, 这棵新的二叉树根结点的权值为其左、右子树根结点权值之和。

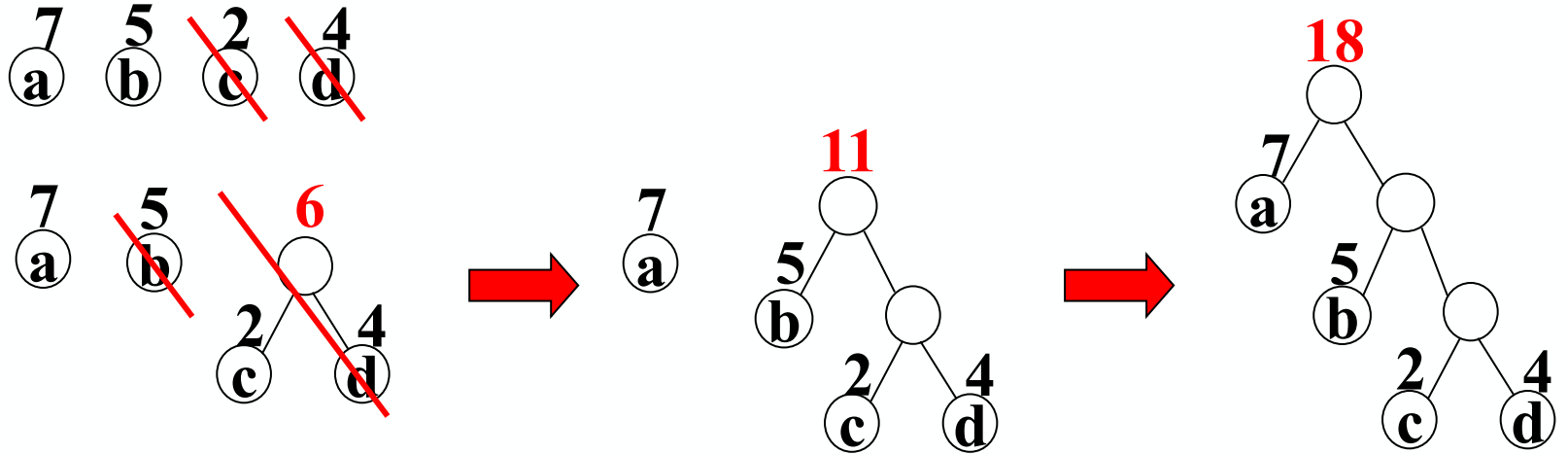
(3) 在集合 F 中删除作为左、右子树的两棵二叉树, 并将新建立的二叉树加入到集合 F 中。

(4) 重复(2)、(3)两步, 当 F 中只剩下一棵二叉树时, 这棵二叉树便是所要建立的哈夫曼树。

注意: 构造哈夫曼树时, 规定 $F=\{T_1, T_2, \dots, T_n\}$ 中**权值小的二叉树**作为新构造的二叉树的**左子树**, **权值大的二叉树**作为新构造的二叉树的**右子树**。

哈夫曼树的构造过程

【例】有4个结点a、b、c、d，已知权值集合 $W=\{7,5,2,4\}$ 。试构造一棵哈夫曼树。



- 哈夫曼树的结点的**度为0或2**，没有度为1的结点。
- 包含 n 棵树的森林要经过 **$n-1$ 次合并**才能形成哈夫曼树，共产生 $n-1$ 个新结点；
- n 棵树构造哈夫曼，则度为0的结点数有 n 个，度为2结点数为 $n-1$ 个
- 包含 n 个叶子结点的哈夫曼树中共有 **$2n-1$** 个结点。

【练习15】 设有13个值，用它们组成一棵哈夫曼树，则该哈夫曼树共有（ ）个结点。

A. 13

B. 12

C. 26

D. 25

答案：D

解释：具有 n 个叶子结点的哈夫曼树共有 $2n-1$ 个结点。 $2*13-1=25$

【练习16】 设哈夫曼树中有199个结点，则该哈夫曼树中有（ ）个叶子结点。

A. 99

B. 100

C. 101

D. 102

答案： B

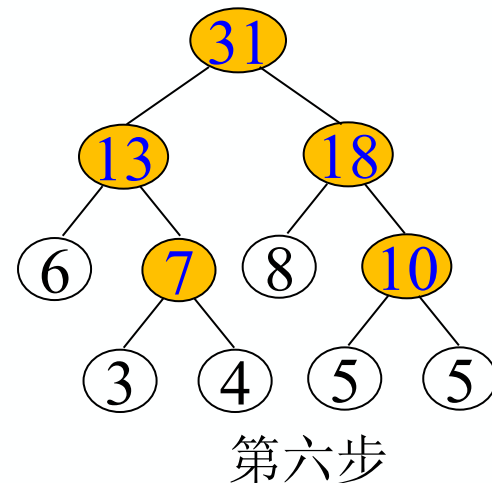
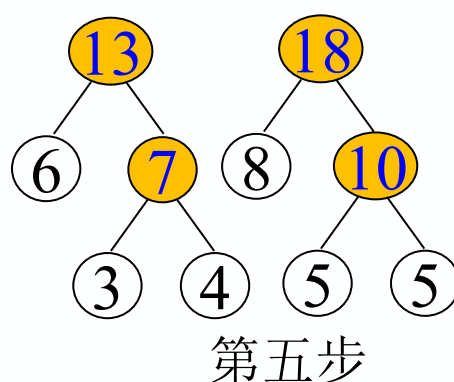
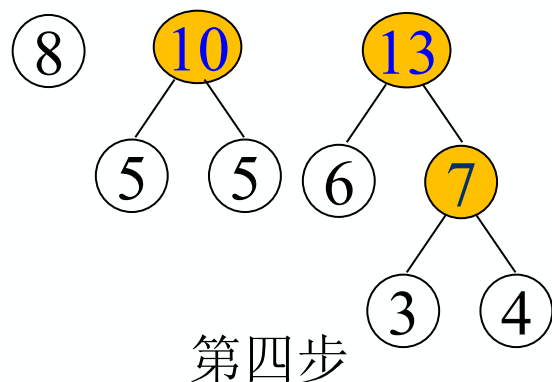
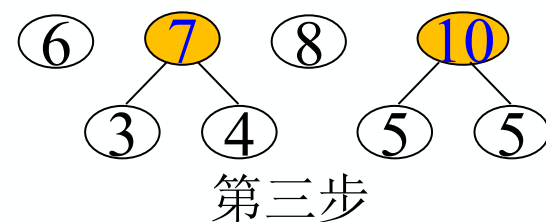
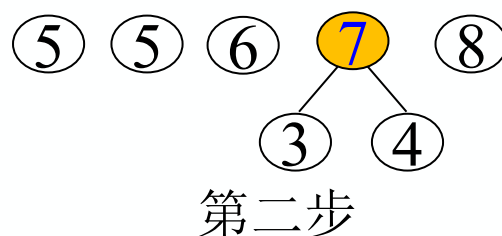
解释： 在哈夫曼树中没有度为1的结点，只有度为0（叶子结点）和度为2的结点。

$2n_0 - 1 = 199$ ，得到 $n_0 = 100$ 。

哈夫曼树的构造过程

【例】已知权值集合 $W=\{8, 3, 4, 6, 5, 5\}$ ，试构造Huffman树。

所构造的Huffman树的 $WPL=6\times 2+3\times 3+4\times 3+8\times 2+5\times 3+5\times 3=79$



Huffman树的构造过程

* 除已有权值结点以外，所有新生成的结点橙色背景标注

已知 $w=(5,29,7,8,14,23,3,11)$ ，试构造一棵哈夫曼树，计算WPL

⑤ ②⑨ ⑦ ⑧ ⑭ ⑳ ③ ⑪

第一步

②⑨ ⑦ ⑧ ⑭ ⑳ ⑪ ⑧

第二步

②⑨ ⑧ ⑭ ⑳ ⑪ ⑮

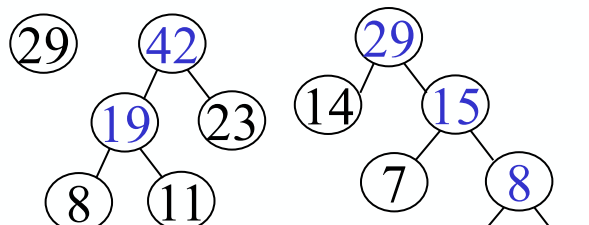
第三步

②⑨ ⑭ ⑳ ⑮ ⑮

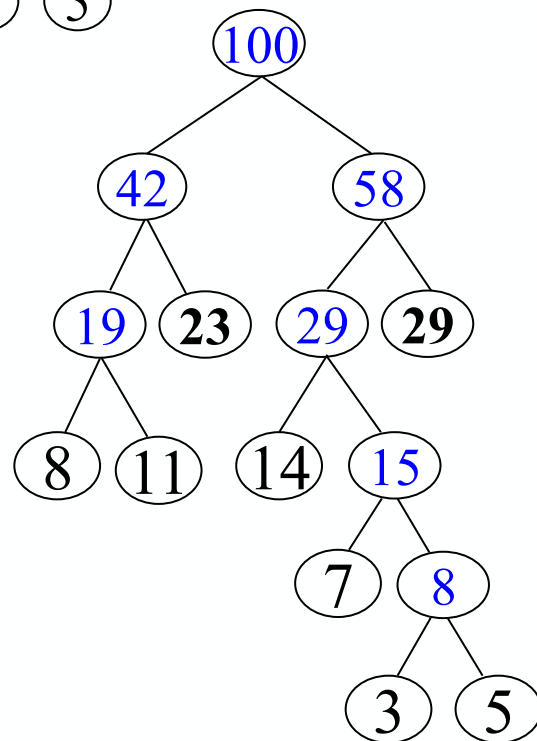
第四步

②⑨ ⑳ ⑮ ⑮ ⑮

第五步

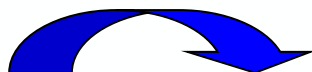
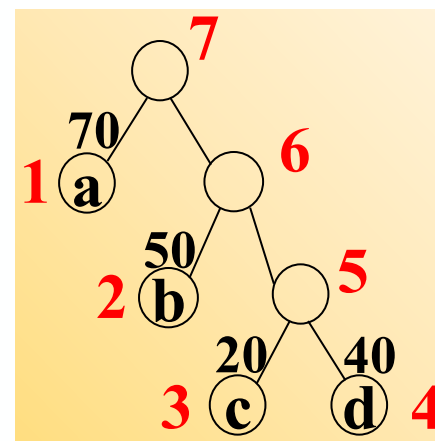
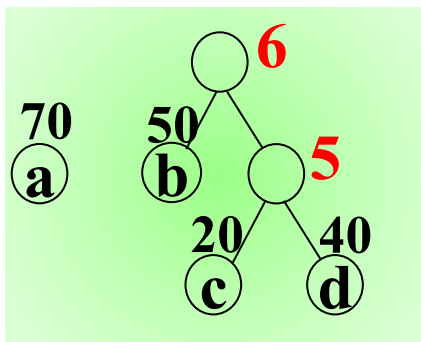
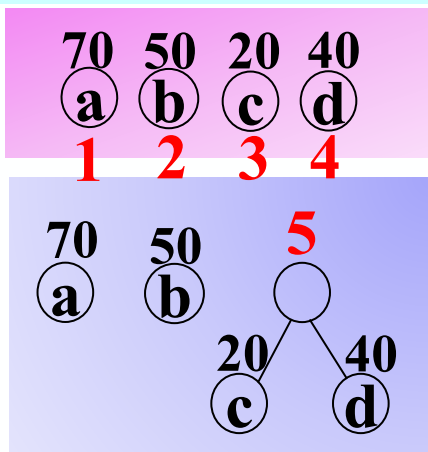


第六步



$$\begin{aligned} \text{WPL} &= 8 \times 3 + 11 \times 3 + 23 \times 2 + 14 \times 3 \\ &\quad + 7 \times 4 + 3 \times 5 + 5 \times 5 + 29 \times 2 \\ &= 271 \end{aligned}$$

【例】 设 $w=\{70,50,20,40\}$ ，试构造哈夫曼树，并给出其构造过程中存储结构HT的初始状态和终结状态。



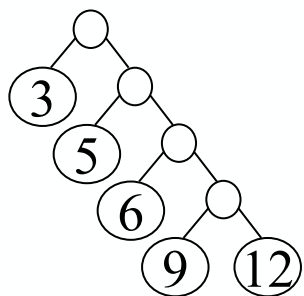
	weight	parent	lch	rch
1	70	0	0	0
2	50	0	0	0
3	20	0	0	0
4	40	0	0	0
5				
6				
7				

HT的初态

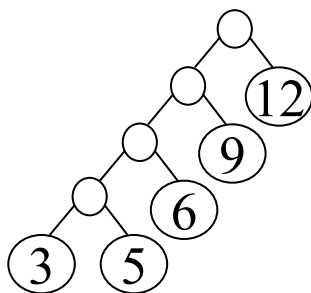
	weight	parent	lch	rch
1	70	7	0	0
2	50	6	0	0
3	20	5	0	0
4	40	5	0	0
5	60	6	3	4
6	110	7	2	5
7	180	0	1	6

HT的终态

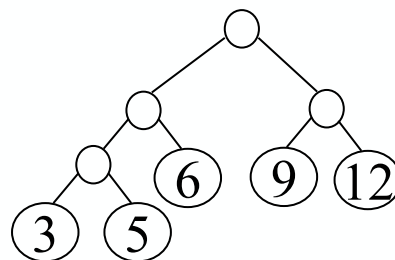
【练习17】 给定整数集合 $\{3, 5, 6, 9, 12\}$ ，与之对应的哈夫曼树是（ ）



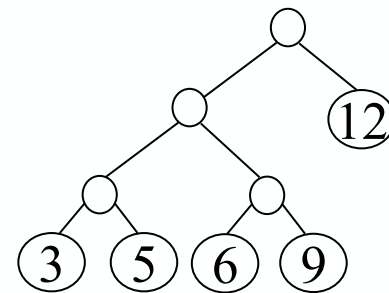
A



B



C



D

答案：C

【练习18】 对 $n(n \geq 2)$ 个权值均不相同的字符构成哈夫曼树，关于该树的叙述中，**错误**的是（ ）

- A. 该树一定是一棵完全二叉树
- B. 树中一定没有度为1的结点
- C. 树中两个权值最小的结点一定是兄弟结点
- D. 树中任一非叶结点的权值一定不小于下一层任一结点的权值

答案： A

5.7.3 哈夫曼编码的构造(P165)

在数据通信、数据压缩问题中，主要手段是电报，即把需要传送的数据文件转换成由**二进制字符0、1组成的二进制字符串**，称之为编码。

假设待传送的数据为7位的'A B A C C D A',文件中只包含A,B,C,D四种字符。(A:3个,B:1个,C:2个,D:1个)

(b)最优

下面有两种编码方案，请比较优劣。

(a)等长编码方案

(b)不等长编码方案

'00010010101100'
7*2=14位

字符	编码
A	00
B	01
C	10
D	11

字符	编码
A	0
B	00
C	1
D	01

'000011010'
3*1+1*2+2*1+1*2=9位

'000011010'
AAAA
ABA
BB

编码核心：①频率高的字符尽可能短的编码；
②任一个字符的编码都不是其他字符的编码的前缀。

前缀编码

5.7.3 哈夫曼编码的构造(P166)

哈夫曼编码——使得电文总长最短的前缀码

1、统计字符集中**每个字符**在电文中**出现的平均概率**。

(**概率越大，要求编码越短**)

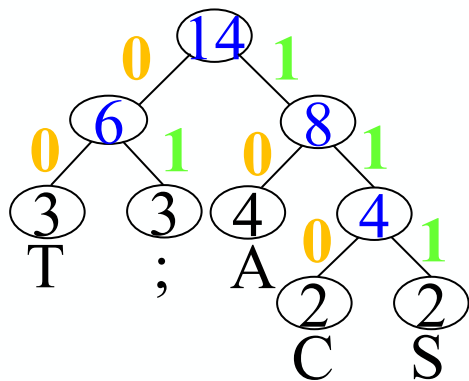
2、利用哈夫曼树的特点：权越大的叶子离根越近；将每个字符的**概率值作为权值，构造哈夫曼树**。则概率越大的结点，路径越短。

3、在哈夫曼树的**每个分支上标上0或1**：

结点的**左分支标0，右分支标1**

把根到每个叶子的路径上的标号连接起来，作为该叶子代表**字符的编码**。

假设待要传输的字符集 $D=\{C,A,S,T,;\}$ ，字符出现频率 $w=\{2,4,2,3,3\}$



T	00
;	01
A	10
C	110
S	111

给定电文ACT;

唯一编码：101100001

反之可倒推。

5.7.3 哈夫曼编码的构造(P166)

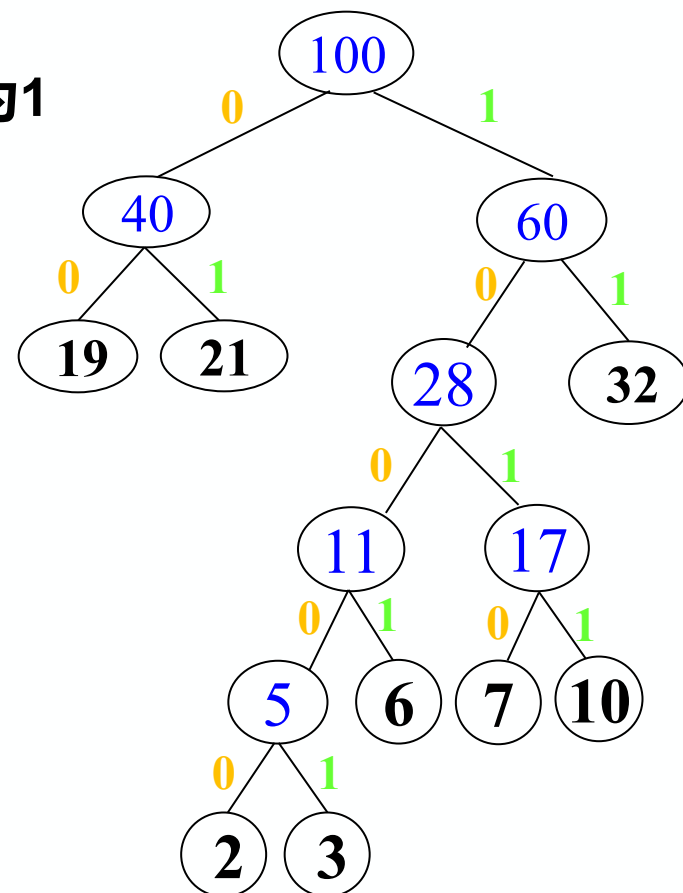
【例】 假设用于通信的电文仅由8种字母组成，字母在电文中出现的频率分别为0.07， 0.19， 0.02， 0.06， 0.32， 0.03， 0.21， 0.10。试为这8个字母设计哈夫曼编码。

答案 (1)先将概率放大100倍， $w=\{7,19,2,6,32,3,21,10\}$

(2)构造哈夫曼树

(3)将树的左分支标记为0， 右分支标记为1

字母编号	出现频率	对应编码
1	0.07	1010
2	0.19	00
3	0.02	10000
4	0.06	1001
5	0.32	11
6	0.03	10001
7	0.21	01
8	0.10	1011



$$WPL=(0.19+0.21+0.32)*2+(0.02+0.03)*5+(0.06+0.07+0.10)*4=2.61$$

【练习19】 根据使用频率为5个字符设计的哈夫曼编码不可能是（ ）。

A. 111,110,10,01,00

B. 000,001,010,011,1

C. 100,11,10,1,0

D. 001,000,01,11,10

答案： C

解释： C中， 100和10冲突， 11和1冲突， 即一个结点既是叶子结点又是内部结点， 哈夫曼树中不可能出现这种情况。

练习一下

1. 树最适合用来表示 ()

- A. 有序数据元素
- B. 元素之间具有分支层次关系的数据
- C. 无序数据元素
- D. 元素之间无联系的数据

2. 树T的层号表示为1a, 2b, 3d, 3e, 2c, 对应于下面选择的 ()

- A. $1a(2b(3d, 3e)), 2c$
- B. $a((b, d, e), c)$
- C. $a(b(d, e), c)$
- D. $a(b, d(e), c)$

3. 如果结点A有3个兄弟, 而且B是A的双亲, 则B的度是_____。

练习一下

4. 假定一棵二叉树的结点数为18个，则它的最小高度为____，它的最大高度为____。
5. 在一棵二叉树中第五层上的结点数最多为____。
6. 在一棵具有五层的满二叉树中，结点总数为____。
7. 假定一棵二叉树的广义表表示为 $A(B(, D), C(E(G), F))$ ，则该树的深度为____，度为0的结点数为____，度为1的结点数为____，度为2的结点数为____。

练习一下

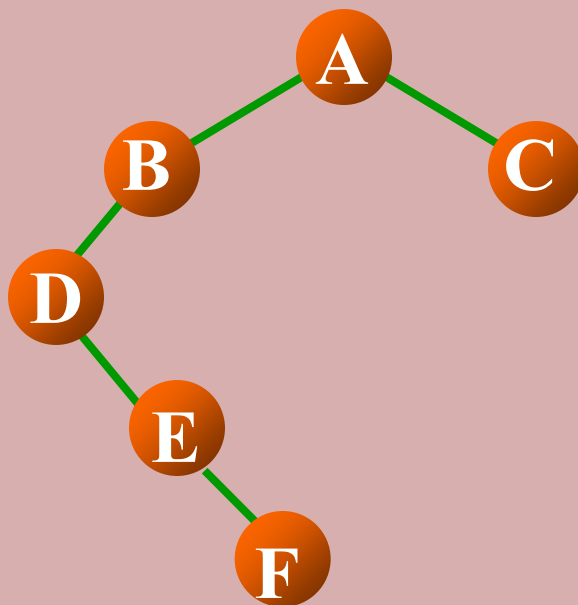
1. 二叉树的先序遍历序列中，任意一个结点均处在其孩子结点的前面，这种说法（ ）。
A. 正确 B. 错误
2. 任何一棵二叉树的叶子结点在先序、中序、后序遍历序列中的相对次序（ ）。
A. 发生改变 B. 不发生改变 C. 不能确定 D. 都不对
3. 二叉树的先序遍历序列是BEFCGDH，中序序列是FEBGCHD，则它的后序序列是（ ），层次序列是（ ）。
A、 EFGHBCD B、 FEGHDCB C、 BCDEFGH
D、 EFBGCHD E、 BECFGDH F、 FEGBHDC

练习一下

4. 已知一棵二叉树的先序遍历结果为ABCDEF, 中序遍历结果为CBAEDF, 则后序遍历的结果为 ()。
- A. CBEFDA B. FEDCBA C. CBEDFA D. 不定
5. 已知某二叉树的后序遍历序列是dabec, 中序遍历序列是debac, 它的先序遍历是 ()。
- A. acbed B. decab C. cedab D. cedba

练习一下

6. 一棵二叉树如图所示，求对此树进行先序、中序、后序遍历时得到的结点序列。

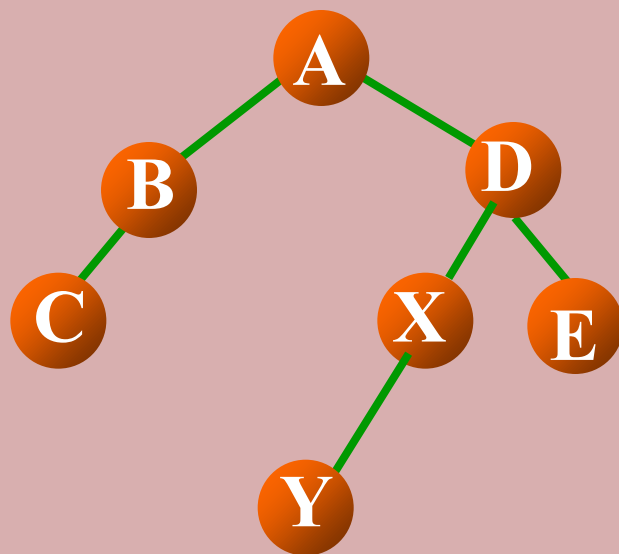


练习一下

1. 引入二叉线索树的目的是 ()
A. 加快查找结点的前驱或后继的速度
B. 为了能在二叉树中方便的进行插入与删除
C. 为了能方便的找到双亲 D. 使二叉树的遍历结果唯一
2. 线索二叉树是一种 () 结构。
A. 逻辑 B. 逻辑和存储 C. 物理 D. 线性
3. 在线索二叉树中, T所指结点没有左子树的充要条件是 ()
A. $T \rightarrow \text{left} = \text{NULL}$ B. $T \rightarrow \text{itag} = 1$
C. $T \rightarrow \text{itag} = 1$ 且 $T \rightarrow \text{left} = \text{NULL}$ D. 以上都不对
4. n 个结点的中序线索二叉树上含有的线索数为 ()
A. $2n$ B. $n-1$ C. $n+1$ D. n

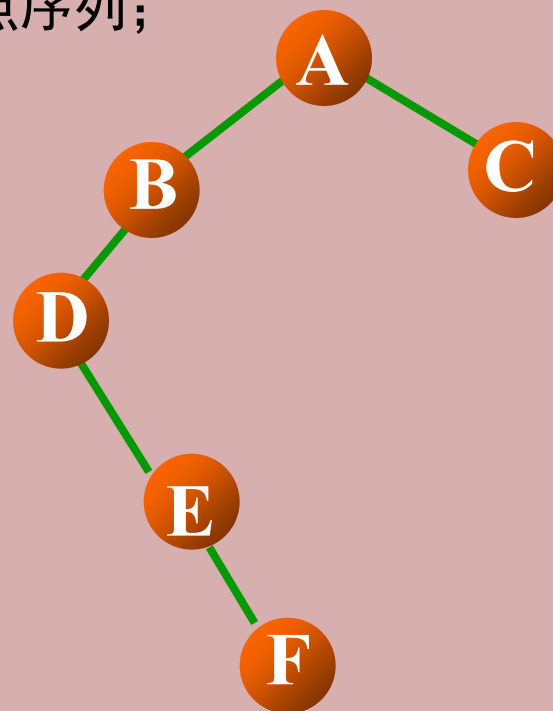
练习一下

5. 将图中的二叉树按中序线索化，结点X的右指针和Y的左指针分别指向（ ）



练习一下

1. 一棵二叉树如图所示：
 - 1) 写出其先序、中序、后序遍历的结点序列；
 - 2) 画出该二叉树的后序线索二叉树。



练习一下

2. 一棵二叉树的结点数据采用顺序存储结构如图所示：

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
E	A	F		D		H			C				G	I					B

- 1) 画出该二叉树；
- 2) 写出结点值为D的双亲结点及左、右子树；
- 3) 将此二叉树还原为森林。

练习一下

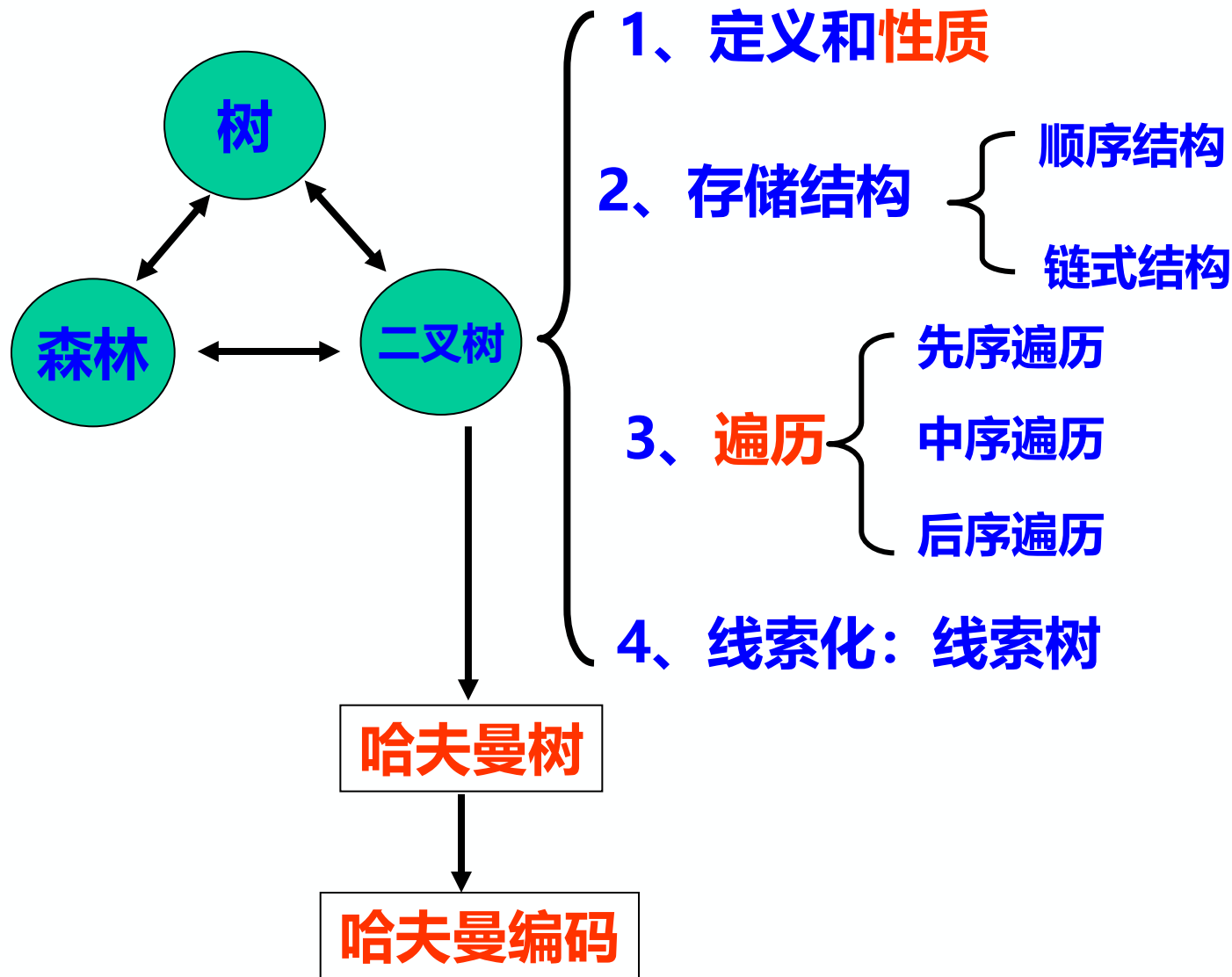
1. 若以 {4, 5, 6, 7, 8} 作为叶子结点的权值构造哈夫曼树，则其带权路径长度是_____。
2. 有权值 $W = \{3, 9, 6, 2, 5\}$ ，则所建 Huffman 树的树高是 (1)。
3. 有一份电文中共使用 6 个字符：a, b, c, d, e, f，它们的出现频率依次为 2, 3, 4, 7, 8, 9，试构造一棵哈夫曼树，字符 c 的编码是 (2)。

练习一下

4. 假设用于通信的电文仅由A、B、C、D、E、F、G、H 8个字母组成，字母在电文中出现的频率分别为0.07, 0.19, 0.02, 0.06, 0.32, 0.03, 0.21, 0.10。

试为这8个字母设计哈夫曼编码。

5. 设有正文MNO PPPOPMMPOPOPPOPNP，字符集为M, N, O, P，设计一套二进制编码，使得上述正文的编码最短。



1. 掌握二叉树的基本概念、**性质**和存储结构
2. 熟练掌握二叉树的**先、中、后序遍历方法**
3. 了解**线索化**二叉树的思想
4. 了解：森林与二叉树的转换
5. 熟练掌握：**哈夫曼树**的实现方法、构造**哈夫曼编码**的方法